

**ҚР ОҚУ-АҒАРТУ МИНИСТРЛІГІ
МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РК
АЛМАТЫ ҚАРЖЫ-ЭКОНОМИКАЛЫҚ КОЛЛЕДЖІ
АЛМАТИНСКИЙ ФИНАНСОВО-ЭКОНОМИЧЕСКИЙ
КОЛЛЕДЖ**

«Қорғауға рұқсат»
Колледж директоры
Алимова К.У.

«_____» _____ 2026 ж., хаттама № _____

ДИПЛОМДЫҚ ЖҰМЫС

**Тақырыбы «Конференция залдар ғимаратындағы іс-шаралар мен
конференцияларды тіркеу веб-жүйесі.»**

Мамандық 06130100 Бағдарламалық қамтамасыз ету (түрлері бойынша)

Біліктілік 4S06130105 - Ақпараттық жүйелер технигі

Орындады: _____ Жанұзақ Ә.Т.

Ғылыми жетекші: _____ Тұрғантай А.Ә.

Алматы, 2026

АНДАТПА

Бұл дипломдық жұмыста іс-шараларға қатысушыларды тіркеу, конференц-залдарды брондау, клиенттік өтінімдерді өңдеу және әкімшілік бақылауды автоматтандыруға арналған ORDA Smart Event System веб-жүйесін әзірлеудің теориялық, жобалық және тәжірибелік негіздері қарастырылды. Жоба Node.js, Express.js, MongoDB, Mongoose, JWT авторизациясы және HTML/CSS/JavaScript негізіндегі клиенттік интерфейс арқылы іске асырылды.

Зерттеу барысында іс-шараларды басқару саласындағы цифрландырудың маңызы, пайдаланушы рөлдері, клиент-сервер архитектурасы, REST API, деректер қоры модельдері және интерфейстің көптілділігі талданды. Жүйеде қатысушы тіркелуі, іс-шара каталогын қарауы, орынға жазылуы, залға өтінім беруі, төлем мәртебесін бақылауы және жеке кабинетте өз өтінімдерін басқаруы мүмкін. Әкімші іс-шаралар, залдар, өтінімдер, статистика және техникалық қолдау хабарламаларын басқарады.

Жұмыстың нәтижесі ретінде оқу орындарына, конференц-орталықтарға және ұйымдарға бейімделетін, кеңейтуге ашық, практикалық маңызы бар веб-платформа ұсынылды.

Жұмыстың кеңейтілген мәтіндік бөлігінде қазіргі веб-жүйелерге қойылатын талаптар, іс-шараларды басқару платформаларының даму бағыты, қолданылған технологиялардың өзара байланысы және қауіпсіздік аспектілері тереңірек талданды. Әсіресе frontend пен backend арасындағы REST API арқылы дерек алмасу, MongoDB базасында құжаттық модельдерді ұйымдастыру, авторизация токендерінің рөлі, пайдаланушы сценарийлері және әкімші панелінің практикалық маңызы жеке қарастырылды.

Зерттеу нәтижелері ORDA Smart Event System жүйесінің тек демонстрациялық веб-қосымша емес, нақты ұйымның күнделікті жұмыс процесіне бейімделе алатын ақпараттық жүйе екенін көрсетеді. Платформада модульдік архитектура, рөлдік қолжетімділік, адаптивті интерфейс, көптілділік және кеңейтілетін деректер моделі қолданылғандықтан, оны кейін төлем жүйелерімен, QR check-in қызметімен, аналитикалық есептермен және хабарлама сервистерімен толықтыруға болады.

АННОТАЦИЯ

В данной дипломной работе рассмотрены теоретические, проектные и практические аспекты разработки веб-системы ORDA Smart Event System, предназначенной для регистрации участников мероприятий, бронирования конференц-залов, обработки клиентских заявок и административного контроля. Проект реализован на базе Node.js, Express.js, MongoDB, Mongoose, JWT-авторизации и клиентского интерфейса HTML/CSS/JavaScript.

В работе проанализированы цифровизация управления мероприятиями, пользовательские роли, клиент-серверная архитектура, REST API, модели базы данных и многоязычный интерфейс. Система позволяет пользователю просматривать каталог событий, регистрироваться на мероприятия, отправлять заявки на аренду зала и отслеживать статусы в личном кабинете.

Администратор управляет мероприятиями, залами, заявками, статистикой и сообщениями технической поддержки.

Дополнительно раскрыты вопросы архитектуры клиент-серверного взаимодействия, безопасности пользовательских данных, масштабируемости облачной базы данных и практического применения системы в учебных организациях, конференц-центрах и бизнес-площадках. Особое внимание уделено тому, как отдельные модули системы связаны между собой: каталог мероприятий, бронирование залов, личный кабинет, административная панель, календарь, карта помещений и техническая поддержка образуют единый рабочий процесс.

ABSTRACT

This diploma thesis presents the theoretical, design, and practical aspects of ORDA Smart Event System, a web platform for event registration, conference hall booking, client request processing, and administrative management. The project uses Node.js, Express.js, MongoDB, Mongoose, JWT authorization, and a HTML/CSS/JavaScript frontend.

The study analyzes digital event management, user roles, client-server architecture, REST API design, database models, and multilingual user interface decisions. The resulting system supports event publishing, participant registration, hall booking requests, personal cabinet workflows, administrator dashboards, and support messages.

The extended thesis also explains the practical value of the system, the interaction between frontend and backend layers, the database structure, security considerations, responsive interface design, and further development opportunities. The project demonstrates how a local organization can replace fragmented spreadsheets, manual registration lists, and separate communication channels with one integrated web platform.

МАЗМҰНЫ

КІРІСПЕ

1 ІС-ШАРАЛАРДЫ БАСҚАРУ ВЕБ-ЖҮЙЕЛЕРІНІҢ ТЕОРИЯЛЫҚ НЕГІЗДЕРІ

- 1.1 Цифрландыру жағдайындағы іс-шараларды ұйымдастыру ерекшеліктері
- 1.2 Қатысушыларды тіркеу, залдарды брондау және пайдаланушы тәжірибесі
- 1.3 Ұқсас платформаларға шолу және ORDA жүйесінің орны
- 1.4 Технологиялық стек және қауіпсіздік негіздемесі

2 ORDA SMART EVENT SYSTEM ЖҮЙЕСІН ЖОБАЛАУ

- 2.1 Жүйеге қойылатын талаптар және пайдаланушы сценарийлері
- 2.2 Клиент-сервер архитектурасы және деректер ағыны
- 2.3 Деректер қоры құрылымы, API логикасы және рөлдік қолжетімділік
- 2.4 Интерфейсті жобалау, көптілділік және визуалдық стиль

3 ЖҮЙЕНІ ІСКЕ АСЫРУ, ТЕКСЕРУ ЖӘНЕ ТИІМДІЛІГІН БАҒАЛАУ

- 3.1 Клиенттік бөліктің негізгі модульдерін жүзеге асыру
- 3.2 Серверлік бөлікті, брондау логикасын және әкімші модулін іске асыру
- 3.3 Тестілеу нәтижелері және анықталған тәуекелдерді талдау
- 3.4 Жобаның практикалық және экономикалық тиімділігі

ҚОРЫТЫНДЫ

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР

ҚОСЫМША А. Бағдарлама құрылымы және негізгі код үзінділері

ҚОСЫМША Б. Қойылатын скриншоттар тізімі

KIPICPE

Қазіргі уақытта іс-шараларды ұйымдастыру саласы қарқынды түрде цифрлануда. Конференциялар, семинарлар, оқу тренингтері, бизнес-форумдар және гибриді форматтағы кездесулер қатысушылардың алдын ала тіркелуін, орын санын бақылауды, залдардың бос уақытын тексеруді, төлем және өтінім мәртебесін қадағалауды талап етеді. Егер бұл әрекеттер қолмен немесе бөлек кестелер арқылы жүргізілсе, ұйымдастырушы уақыт жоғалтады, деректер қайталанатын және қатысушыға берілетін кері байланыс баяулайды.

ORDA Smart Event System жобасы осы мәселені шешуге бағытталған. Жүйе веб-платформа ретінде жасалып, бір интерфейсте іс-шаралар каталогын, қатысушыларды тіркеуді, конференц-залдарды брондауды, күнтізбелік бақылауды, әкімші панелін және техникалық қолдау хабарламаларын біріктіреді. Платформада пайдаланушы мен әкімші рөлдері бөлінген, ал деректер MongoDB базасында сақталады.

Дипломдық жұмыстың мақсаты - іс-шараларға тіркеу және конференц-залдарды басқару процесін автоматтандыратын ORDA Smart Event System веб-жүйесінің теориялық негіздерін сипаттау, архитектурасын жобалау және практикалық іске асырылуын негіздеу.

Осы мақсатқа жету үшін төмендегідей міндеттер қойылды:

- іс-шараларды ұйымдастыру мен брондау процестерін цифрландырудың маңызын талдау;
- пайдаланушы, қатысушы және әкімші сценарийлерін анықтау;
- жүйенің функционалдық және функционалдық емес талаптарын қалыптастыру;
- клиент-сервер архитектурасын, REST API және деректер қоры модельдерін жобалау;
- Node.js, Express.js және MongoDB негізінде серверлік бөлікті іске асыру;
- HTML, CSS және JavaScript негізінде көптілді веб-интерфейс әзірлеу;
- брондау, өтінім мәртебесі, төлем, күнтізбе және техникалық қолдау модульдерін сипаттау;
- жүйені функционалдық тексеруден өткізіп, практикалық тиімділігін бағалау.

Зерттеу нысаны - іс-шараларды жариялау, қатысушыларды тіркеу және конференц-залдарды брондау процесі. Зерттеу пәні - осы процесті автоматтандыруға арналған веб-жүйенің функционалдық, архитектуралық және интерфейстік шешімдері.

Зерттеу әдістері ретінде салыстырмалы талдау, жүйелік жобалау, UML элементтеріне негізделген сценарийлік талдау, деректер моделін құру, REST API жобалау, бағдарламалық іске асыру және функционалдық тестілеу әдістері қолданылды.

Жұмыстың практикалық маңызы - ORDA жүйесін оқу орындарында, конференц-залдарда, бизнес орталықтарында, курстар мен семинарлар өткізетін

ұйымдарда қолдануға болады. Жүйе ұйымдастырушыға іс-шараны жариялау, орын санын бақылау, қатысушы өтінімдерін өңдеу және залдардың бос уақытын бір ортадан басқаруға мүмкіндік береді.

Дипломдық жұмыс кіріспеден, үш негізгі бөлімнен, қорытындыдан, пайдаланылған әдебиеттер тізімінен және қосымшалардан тұрады. Бірінші бөлімде тақырыптың теориялық негіздері қарастырылды. Екінші бөлімде жүйенің жобалық шешімдері берілді. Үшінші бөлімде бағдарламалық іске асыру, тестілеу және тиімділік нәтижелері сипатталды.

Тақырыптың өзектілігі қазіргі ұйымдардың ақпаратпен жұмыс істеу тәсілінің өзгеруімен тікелей байланысты. Бұрын іс-шараны өткізу үшін қатысушылардың тізімі, залдың бос уақыты, техникалық жабдық, төлем туралы ақпарат және кері байланыс бөлек құжаттарда немесе мессенджер хаттарында жүргізілсе, қазіргі веб-жүйелер бұл деректерді бір ортаға жинауға мүмкіндік береді. Мұндай тәсіл ұйымдастырушының жұмысын жеңілдетіп қана қоймай, шешім қабылдау сапасын да арттырады, себебі әкімші нақты уақытқа жақын деректерге сүйенеді.

Конференция залдары мен іс-шараларды басқару саласында автоматтандырудың маңызы ерекше. Бір зал бірнеше топқа қажет болуы мүмкін, бір күнде бірнеше кездесу жоспарлануы ықтимал, ал қатысушылар саны зал сыйымдылығынан асып кетпеуі тиіс. Егер осы шарттар қолмен тексерілсе, адами фактордың әсері артады. ORDA Smart Event System жобасы дәл осындай қайталанатын операцияларды жүйелік түрде орындауға бағытталған: пайдаланушы өтінім береді, сервер деректерді тексереді, базаға сақтайды, ал әкімші мәртебені өзгертіп, процесті бақылайды.

Жұмыстың ғылыми-тәжірибелік маңыздылығы веб-қосымшаны тек техникалық өнім ретінде емес, нақты басқару процесін оңтайландыратын ақпараттық жүйе ретінде қарастыруында. Мұнда бағдарламалық код, интерфейс, деректер қоры және пайдаланушы сценарийлері бір-бірінен бөлек емес, біртұтас шешім ретінде талданады. Сондықтан дипломдық жұмыста тек қолданылған технологиялар аталып қана қоймай, олардың не үшін таңдалғаны, қандай міндетті шешетіні және жүйенің болашақтағы дамуына қалай әсер ететіні түсіндіріледі.

Зерттеу барысында іс-шараларды басқару платформаларының негізгі даму үрдістері де ескерілді. Қазіргі сервистер пайдаланушыға жылдам тіркелу, мобильді құрылғыдан қолжетімділік, онлайн және офлайн форматтарды қатар қолдау, жеке кабинет, төлем мәртебесі, автоматты хабарламалар және аналитикалық есептер ұсынуға тырысады. ORDA жобасы осы үрдістерді дипломдық жоба деңгейінде бейімдеп, жергілікті ұйымға түсінікті әрі басқаруға ыңғайлы форматта іске асырады.

Жүйені әзірлеу кезінде функционалдық талаптармен қатар функционалдық емес талаптар да қарастырылды. Оларға қауіпсіздік, жауап беру жылдамдығы, интерфейснің ыңғайлылығы, код құрылымының түсініктілігі, деректердің сақталу тұрақтылығы, болашақта кеңейту мүмкіндігі және әртүрлі экран өлшемдеріне бейімделу жатады. Бұл талаптар жобаның

сапасын тек экранда көрінетін нәтиже арқылы емес, оның ішкі архитектурасы арқылы да бағалауға мүмкіндік береді.

Кіріспеде белгіленген мақсат пен міндеттер дипломдық жұмыстың барлық бөлімдерімен байланысады. Бірінші бөлімде мәселенің теориялық негізі ашылып, ұқсас платформалармен салыстыру жүргізіледі. Екінші бөлімде ORDA жүйесінің құрылымы, деректер ағыны және API логикасы сипатталады. Үшінші бөлімде нақты іске асыру, модульдердің жұмысы, тестілеу нәтижелері, тәуекелдер және практикалық тиімділік талданады. Осылайша жұмыс теориядан практикалық нәтижеге дейінгі толық циклді қамтиды.

1 ІС-ШАРАЛАРДЫ БАСҚАРУ ВЕБ-ЖҮЙЕЛЕРІНІҢ ТЕОРИЯЛЫҚ НЕГІЗДЕРІ

1.1 Цифрландыру жағдайындағы іс-шараларды ұйымдастыру ерекшеліктері

Іс-шараларды ұйымдастыру - бірнеше өзара байланысты әрекеттерден тұратын процесс. Оған бағдарлама жасау, қатысушыларды шақыру, тіркеу, орын санын шектеу, залды таңдау, техникалық жабдықты дайындау, төлемді белгілеу, қатысу мәртебесін тексеру және іс-шара аяқталғаннан кейін статистика жинау кіреді. Дәстүрлі тәсілде бұл әрекеттер телефон, электрондық пошта, қағаз тізім немесе жеке кестелер арқылы орындалады.

Мұндай тәсіл шағын кездесулер үшін жеткілікті болуы мүмкін, бірақ қатысушылар саны өскен кезде қателік ықтималдығы артады. Бір қатысушы бірнеше рет тіркелуі, бос орын саны дұрыс есептелмеуі, зал басқа іс-шарамен қатар брондалып кетуі немесе әкімшіге өтінім мәртебесін қолмен жаңарту қажет болуы мүмкін. Сондықтан іс-шара мен залды басқару жүйесі тек ақпараттық сайт емес, бизнес-процесті автоматтандыратын құрал болуы тиіс.

Цифрландырудың негізгі артықшылығы - деректердің бір орталықта сақталуы. Қатысушы тіркелген кезде оның ақпараты бірден деректер қорына түседі, әкімші өтінімді көреді, жүйе орын санын жаңартады және күнтізбе жүктемені көрсетеді. Бұл ұйымдастырушының жұмысын жеңілдетіп қана қоймай, пайдаланушыға да сенімді тәжірибе береді.

ORDA Smart Event System осы логикаға сүйенеді. Жүйеде іс-шара, зал, пайдаланушы, өтінім және техникалық қолдау хабарламасы жеке деректер моделі ретінде қарастырылған. Мұндай бөлу бағдарламаны кеңейтуге және әр модульді тәуелсіз жетілдіруге мүмкіндік береді.

Іс-шараларды басқару процесін жүйелік тұрғыдан қарастырғанда, оның бірнеше кезеңнен тұратынын көруге болады. Бірінші кезеңде ұйымдастырушы іс-шара идеясын, тақырыбын, өткізу форматын, уақытын және орнын анықтайды. Екінші кезеңде қатысушыларды шақыру және тіркеу жүргізіледі. Үшінші кезеңде зал, жабдық, төлем, техникалық қолдау және қатысу мәртебесі бақыланады. Соңғы кезеңде статистика жиналып, келесі іс-шараларға арналған тәжірибе қалыптасады.

Цифрлық жүйе осы кезеңдердің әрқайсысында деректердің үздіксіз қозғалысын қамтамасыз етеді. Мысалы, іс-шара жарияланған сәтте ол бірден каталогта көрінеді, қатысушы тіркелгенде бос орын саны өзгереді, әкімші панелінде жаңа өтінім пайда болады, ал жеке кабинетте пайдаланушы өзінің қатысу мәртебесін бақылай алады. Мұндай байланыс деректерді қайта-қайта енгізу қажеттілігін азайтады және бір ақпараттың бірнеше жерде әртүрлі сақталу қаупін төмендетеді.

Іс-шара саласындағы цифрландырудың тағы бір маңызды бағыты - гибриді форматтарды қолдау. Қазіргі конференциялар тек залда өтпейді: кейбір қатысушылар онлайн қосылады, кейбірі офлайн қатысады, ал

ұйымдастырушыға екі форматтың да деректерін басқару қажет болады. Сондықтан жүйеде offline, online және hybrid форматтарының болуы интерфейс үшін ғана емес, деректер моделін дұрыс жобалау үшін де маңызды. Әр форматтың өзіндік өрістері мен тексеру шарттары болуы мүмкін.

Дәстүрлі кестелік есепке алу мен веб-жүйенің айырмашылығы деректердің белсенді өңделуінде. Кестеде ақпарат сақталады, бірақ ол автоматты түрде қақтығысты анықтамайды, пайдаланушы рөлін тексермейді және жеке кабинетке нәтижені жібермейді. Ал веб-жүйеде деректер тек сақталмайды, олар бизнес-логика арқылы тексеріледі. ORDA жүйесінде бір пайдаланушының бір іс-шараға қайталанып жазылмауы, зал уақытының қақтығыспауы және мәртебелердің бақылануы осыған мысал бола алады.

Ұйымдастыру процесін автоматтандырудың әлеуметтік әсері де бар. Қатысушы үшін жүйе сенімділік сезімін береді: ол өтінімінің қабылданғанын, төлемнің қандай күйде екенін және қандай іс-шараларға жазылғанын көре алады. Әкімші үшін жүйе бақылауды күшейтеді: барлық өтінім бір жерде жиналады, статистика тез табылады, ал техникалық қолдау хабарламалары жоғалып кетпейді. Осылайша цифрландыру тек техникалық жаңарту емес, қызмет көрсету мәдениетін жақсарту құралы болып табылады.

ORDA жобасының теориялық негізінде ақпараттық жүйелерді модульге бөлу қағидасы жатыр. Іс-шара, зал, пайдаланушы, брондау және қолдау хабарламасы жеке объект ретінде қарастырылғандықтан, әр объектінің өмірлік циклі бөлек анықталады. Мысалы, іс-шара draft, published немесе closed күйінде болуы мүмкін, ал брондау new, review, pending, approved, rejected, cancelled немесе registered мәртебесіне өтеді. Мұндай модель нақты процесті деректер деңгейінде бейнелейді.

1.2 Қатысушыларды тіркеу, залдарды брондау және пайдаланушы тәжірибесі

Қатысушы үшін веб-жүйенің басты құндылығы - әрекеттің түсінікті және жылдам орындалуы. Пайдаланушы сайтқа кіріп, қажетті іс-шараны іздейді, категория немесе формат бойынша сүзгіден өткізеді, толық ақпаратты қарайды және бір батырма арқылы тіркеледі. Егер тіркелу үшін күрделі форма немесе түсініксіз навигация қажет болса, жүйенің пайдалану тиімділігі төмендейді.

Залдарды брондау сценарийі күрделірек, себебі мұнда күн, уақыт, ұзақтық, қатысушылар саны, қосымша қызметтер, жабдық және төлем мәртебесі ескеріледі. Сонымен қатар жүйе бір залдың бір уақытта екі өтінімге бекітілуін болдырмауы керек. Сондықтан брондау модулінде қолжетімділік тексерісі және уақыт аралықтарының қақтығысын анықтау алгоритмі маңызды орын алады.

Пайдаланушы тәжірибесі тек интерфейс әдемілігімен өлшенбейді. Ол ақпараттың табылуы, батырмалардың болжамдылығы, қате хабарламаларының түсініктілігі, тіл таңдауы, мобильді экранда ыңғайлы көрінуі және жүйенің жауап беру жылдамдығы сияқты факторлардан тұрады. ORDA жобасында осы себепті бүйірлік навигация, іздеу, фильтрлер, көптілді мәтіндер, жарық/қараңғы тақырып және жеке кабинет қарастырылған.

Пайдаланушы тәжірибесін жобалауда әрекеттердің болжамдылығы маңызды орын алады. Егер пайдаланушы іс-шара карточкасында атауды, күнді, форматты, орынды, бағаны және бос орын санын бірден көрсе, ол шешімді тез қабылдайды. Керісінше, маңызды ақпарат бірнеше терезеге бөлініп кетсе немесе батырманың мағынасы түсініксіз болса, тіркелу ықтималдығы төмендейді. Сондықтан ORDA интерфейсінде карточка, фильтр, іздеу және action батырмалары бір сценарийге біріктірілген.

Зал брондау кезінде пайдаланушы тәжірибесі ақпараттың толықтығымен байланысты. Қатысушы саны, жабдық, уақыт, мақсат, төлем әдісі және қосымша қызметтер нақты көрсетілуі керек. Бұл деректер кейін әкімші шешіміне әсер етеді: залдың сыйымдылығы жеткілікті ме, сұралған уақыт бос па, қызметтер қолжетімді ме, төлем мәртебесі қандай. Осы себепті брондау формасы қарапайым көрінгенімен, оның артында толық бизнес-процесс орналасқан.

Жүйеде қате жағдайларын түсінікті көрсету де UX/UI сапасының бір бөлігі болып табылады. Мысалы, пайдаланушы авторизациясыз қорғалған бөлімге кірсе, жүйе оған қайта кіру қажеттігін көрсетуі тиіс. Егер зал уақыты бос болмаса, қате хабарламасы жалпы техникалық мәтін емес, нақты себепті түсіндіруі қажет. Мұндай хабарламалар пайдаланушыны жүйеден алыстатпайды, керісінше келесі әрекетті түсінуге көмектеседі.

Адаптивті интерфейсін рөлі де жоғары. Іс-шараға қатысушы сайтқа көбіне телефоннан кіруі мүмкін: ол жолда келе жатып тіркеледі, зал туралы ақпаратты қарайды немесе жеке кабинеттен мәртебені тексереді. Сондықтан интерфейс тек үлкен мониторға емес, мобильді экранға да бейімделуі тиіс. CSS арқылы grid, flex, responsive spacing және ыңғайлы батырма өлшемдері қолданылса, жүйенің қолданылу аясы кеңейеді.

Көптілділік пайдаланушы тәжірибесін кеңейтетін маңызды факторлардың бірі. Қазақстан жағдайында қазақ, орыс және ағылшын тілдеріндегі интерфейс әртүрлі аудиторияға қолжетімділік береді. ORDA жүйесінде мәтіндерді `data-i18n` атрибуттары арқылы бөлу бір HTML құрылымын сақтап, тілдік мазмұнды бөлек басқаруға мүмкіндік береді. Бұл тәсіл кейін жаңа тіл қосу немесе терминдерді түзету кезінде интерфейс кодын толық қайта жазбауға жағдай жасайды.

UX талдауында әкімші тәжірибесін де бөлек бағалау қажет. Әкімші үшін интерфейс әдемі ғана емес, жұмысқа ыңғайлы болуы керек: өтінімдерді жылдам сүзу, мәртебені өзгерту, статистиканы көру, жаңа іс-шара немесе зал қосу, техникалық хабарламаларды бақылау сияқты әрекеттер жиі қайталанады. Сондықтан admin dashboard бөлімінде ақпараттың жинақы орналасуы және басқару батырмаларының айқын болуы жүйенің практикалық құндылығын арттырады.

1.3 Ұқсас платформаларға шолу және ORDA жүйесінің орны

Нарықта іс-шараларды басқаруға арналған түрлі платформалар бар. Олардың ішінде халықаралық деңгейде Eventbrite және Cvent, ал ТМД аумағында TimePad сияқты сервистер белгілі. Бұл платформалар қатысушы

тіркеу, билет сату, іс-шара беттерін жариялау және аналитика мүмкіндіктерін ұсынады. Алайда оқу орны немесе жергілікті конференц-зал үшін олардың кейбірі тым күрделі, кейбірі ақылы, ал кейбірінде залды ішкі кестемен басқару сценарийі шектеулі болуы мүмкін.

ORDA жүйесінің ерекшелігі - ол дипломдық жоба ретінде нақты жергілікті сценарийге бейімделген. Платформада іс-шараға қатысу және залды брондау бір жүйеде біріктірілген. Сонымен қатар әкімші клиенттік өтінімді қарап, бекіте алады, ал бекітілген жеке іс-шара автоматты түрде каталогқа жариялануы мүмкін.

Eventbrite, Cvent және TimePad сияқты платформалар ауқымды нарыққа бағытталғандықтан, олардың функционалы кең және кейде күрделі болады. Eventbrite көбіне іс-шара жариялау, билет сату және қатысушы аудиториясын тарту міндеттеріне ыңғайлы. Cvent корпоративтік деңгейдегі конференция менеджментіне, қонақ үй және ірі event-инфрақұрылыммен интеграцияға бейім. TimePad тіркеу мен билет таратуда ыңғайлы, бірақ жергілікті зал ресурстарын ұйым ішіндегі кестемен басқару барлық жағдайда негізгі сценарий бола бермейді.

ORDA жүйесін салыстырғанда негізгі критерийлер ретінде функционалдық толықтық, жергілікті бейімделу, әкімшіге түсініктілік, техникалық тәуелсіздік, кеңейту мүмкіндігі және оқу жобасына сәйкестік алынды. Коммерциялық платформалар дайын сервис ретінде күшті, бірақ оларды ұйымның ішкі процесіне толық бейімдеу қиын болуы мүмкін. Ал дипломдық жоба ретінде жасалған ORDA нақты талаптарға қарай өзгертіледі, код құрылымы ашық, деректер моделі кеңейтуге мүмкіндік береді.

Ұқсас платформаларды талдау ORDA жобасының артықшылығы мен шектеулерін қатар көруге мүмкіндік береді. Артықшылықтарына бір ортада іс-шара мен зал брондауын біріктіру, қарапайым әкімші панелі, көптілді интерфейс, MongoDB арқылы икемді дерек сақтау және нақты оқу процесіне бейімделу жатады. Шектеулеріне толық төлем шлюзінің, email/SMS хабарламаларының және күрделі аналитиканың әзірше демонстрациялық деңгейде болуы кіреді. Бұл шектеулер жобаның болашақ даму жоспарын анықтайды.

Салыстырмалы талдаудың нәтижесінде ORDA жүйесінің нарықтағы орны шағын және орта ұйымдарға арналған бастапқы автоматтандыру платформасы ретінде анықталады. Ол ірі коммерциялық сервистермен тікелей бәсекелесуді мақсат етпейді, бірақ жергілікті конференц-зал, колледж, оқу орталығы немесе бизнес кеңістігі үшін негізгі процестерді цифрландыруға жеткілікті. Осы тұрғыдан жоба практикалық қолдануға жақын және оқу жұмысы ретінде нақты нәтижеге ие.

ORDA платформасының тағы бір ерекшелігі - жүйені өз инфрақұрылымында орналастыру мүмкіндігі. Коммерциялық сервистерде деректер көбіне сыртқы провайдердің ережелеріне тәуелді болады, ал жеке жасалған жүйеде деректер базасын, сервер ортасын, доменді және қауіпсіздік параметрлерін ұйым өзі бақылай алады. Бұл әсіресе оқу орындары мен шағын

ұйымдар үшін маңызды, себебі қатысушылардың жеке деректері мен ішкі кестелерін басқару жауапкершілігі нақты анықталуы тиіс.

Осы талдау ORDA жүйесін дамытуда қандай бағыттарға көңіл бөлу керектігін көрсетеді. Біріншіден, тіркеу мен брондау логикасы тұрақты болуы керек. Екіншіден, әкімшіге арналған статистика мен экспорт мүмкіндіктері күшейтілуі қажет. Үшіншіден, қауіпсіздік пен қолжетімділік талаптары жүйенің әр қабатында ескерілуі тиіс. Төртіншіден, интерфейс қолданушыны шатастырмай, бірнеше рөлдің қажеттілігін бір ортада қамтуы керек.

1-кесте. Ұқсас платформалар мен ORDA жүйесінің салыстырмасы

Критерий	Eventbrite	TimePad	Cvent	ORDA Smart Event System
Іс-шара жариялау	Қолдайды	Қолдайды	Қолдайды	Қолдайды
Қатысушы тіркеу	Қолдайды	Қолдайды	Қолдайды	Қолдайды
Конференц-зал брондау	Шектеулі	Шектеулі	Кең, бірақ күрделі	Жүйенің негізгі модулі
Жергілікті оқу орнына бейімдеу	Қосымша баптау қажет	Қосымша баптау қажет	Күрделі енгізу қажет	Жоба сценарийіне тікелей бейімделген
Көптілді интерфейс	Қолдайды	Ішінара	Қолдайды	RU/KZ/EN мәтіндері бар
Әкімші өтінімдері	Бар	Бар	Бар	Өтінім, төлем, check-in, қолдау хабарламалары

1.4 Технологиялық стек және қауіпсіздік негіздемесі

ORDA жобасы классикалық веб-архитектура бойынша құрылған: клиенттік бөлік HTML, CSS және JavaScript арқылы жасалады, ал серверлік бөлік Node.js ортасында Express.js фреймворкімен жұмыс істейді. Деректер MongoDB құжаттық базасында сақталып, Mongoose кітапханасы арқылы модельдермен байланысады.

Қауіпсіздік тұрғысынан жүйеде пайдаланушы құпиясөзі bcryptjs арқылы хэштеледі, ал жүйеге кіргеннен кейін JWT токени беріледі. Токен пайдаланушының идентификаторын, рөлін, email мекенжайын және атын қамтиды. Сервердегі middleware қабаты қорғалған маршруттарға тек авторизацияланған пайдаланушыны, ал әкімші маршруттарына тек admin рөлін өткізеді.

HTML, CSS және JavaScript клиенттік қабаттың негізгі технологиялары ретінде таңдалды. HTML құжат құрылымын, бөлімдер мен формаларды анықтайды; CSS визуалды стиль, орналасу, түстер, адаптивті торлар және тақырып ауыстыру логикасын қамтамасыз етеді; JavaScript пайдаланушы әрекеттеріне жауап береді, API сұраныстарын жібереді, алынған деректерді

DOM құрылымына шығарады және интерфейстің күйін басқарады. Бұл үш технология веб-қосымшаның пайдаланушыға көрінетін бөлігін қалыптастырады.

Node.js серверлік JavaScript ортасы ретінде жобаның backend бөлігін құруға мүмкіндік берді. Оның артықшылығы - frontend пен backend бір тілдік экожүйеде жазылады, бұл шағын команда немесе оқу жобасы үшін ыңғайлы. Express.js фреймворкі HTTP маршруттарды, middleware тізбегін, JSON body parser, CORS баптауын, статикалық файлдарды тарату және error handler механизмін қарапайым ұйымдастыруға көмектеседі. Осы арқылы сервер тек файл таратушы емес, толық REST API қызметін атқарады.

MongoDB құжаттық деректер қоры ORDA жобасының объектілік табиғатына сәйкес келеді. Іс-шара, зал, брондау және пайдаланушы құжаттары бірдей қатаң реляциялық кестеге бағынбайды, олардың кейбір өрістері уақыт өте кеңейі мүмкін. Мысалы, іс-шараға translations, tags, agenda, meetingUrl сияқты қосымша өрістер қосылады, ал зал объектісінде equipment және advantages массивтері бар. Mongoose бұл икемділікті сақтай отырып, схема деңгейінде міндетті өрістерді, типтерді және enum мәндерін бақылауға мүмкіндік береді.

REST API жүйенің клиент пен сервер арасындағы келісімін анықтайды. Frontend белгілі endpoint-ке сұраныс жібереді, backend сұранысты тексереді, деректер қорымен жұмыс істейді және JSON форматында жауап қайтарады. Бұл тәсіл интерфейсті сервер логикасынан бөледі: кейін мобильді қосымша немесе басқа frontend жасалса, сол API-ді қайта пайдалануға болады. Сонымен қатар API маршруттарын ашық, авторизацияланған және әкімшіге ғана қолжетімді деп бөлу қауіпсіздік архитектурасын нақтылайды.

Қауіпсіздіктің бірінші деңгейі - парольдерді ашық мәтін түрінде сақтамау. bcryptjs құпиясөзді хэшке айналдырады, сондықтан деректер базасына тікелей қолжетімділік болған жағдайда да бастапқы құпиясөз көрінбейді. Екінші деңгей - JWT арқылы сессияны басқару. Пайдаланушы login жасағанда сервер қол қойылған токен береді, ал қорғалған маршруттарға сұраныс жібергенде frontend оны Authorization header арқылы жібереді. Сервер токенді тексеріп, пайдаланушының рөлі мен құқықтарын анықтайды.

Қауіпсіздіктің үшінші деңгейі - рөлдік қолжетімділік. ORDA жүйесінде қарапайым пайдаланушы өзінің өтінімдерін, профилін және тіркелулерін көре алады, ал әкімші барлық өтінімдер мен статистикаға қол жеткізеді. `adminOnly` middleware әкімші емес пайдаланушының admin маршрутқа кіруін тоқтатады. Бұл тәсіл интерфейсте admin батырмасын жасырумен шектелмейді, себебі нақты қорғаныс сервер деңгейінде орындалады.

CORS баптауы да қауіпсіздік пен орналастыру тұрғысынан маңызды. Жүйе localhost, 127.0.0.1 және GitHub Pages сияқты рұқсат етілген origin-дермен жұмыс істей алады, ал production режимінде бөтен origin-ді шектеуге мүмкіндік бар. Бұл браузер арқылы рұқсатсыз cross-origin сұраныстардың алдын алуға көмектеседі. Сонымен қатар JSON body көлемінің шектелуі серверді тым үлкен сұраныстардан қорғаудың қарапайым тәсілі болып табылады.

Қауіпсіздік архитектурасында деректерді енгізу сапасын бақылау да маңызды. Пайдаланушы формадаң жіберген ақпаратты тек frontend деңгейінде тексеру жеткіліксіз, себебі HTTP сұранысын браузерден тыс құралдар арқылы да жіберуге болады. Сондықтан backend controller-лері міндетті өрістерді, типтерді, enum мәндерін, уақыт форматын, қатысушылар санын және пайдаланушы құқығын қайта тексереді. Бұл тәсіл жүйенің сенімділігін арттырады және қате немесе әдейі бұрмаланған деректердің базаға түсу қаупін төмендетеді.

Жеке деректермен жұмыс істегенде минимизация қағидасын сақтау қажет. ORDA жүйесінде пайдаланушыдан жүйе жұмысына керек негізгі ақпарат қана сұралады: аты, email, телефон, ұйым және қала. Қатысушының артық жеке мәліметін жинамау қауіпсіздік тәуекелін азайтады. Сонымен қатар құпия параметрлерді `.env` файлында сақтау, репозиторийге жарияламау, JWT_SECRET мәнін жеткілікті күрделі ету және MongoDB Atlas connection string-ін қорғау production орналастырудың міндетті шарты болып табылады.

Бұл технологиялық стек дипломдық жоба үшін теңгерімді шешім болып табылады. Ол бір жағынан заманауи веб-әзірлеу тәжірибесіне сәйкес келеді, екінші жағынан шамадан тыс күрделі микросервистік архитектураны қажет етпейді. Жүйе кейін масштабталса, Express API-ді бөлек серверге шығару, MongoDB Atlas кластерін кеңейту, CDN қолдану, кэштеу қосу және frontend-ті жеке hosting-ке орналастыру сияқты бағыттар қарастырылуы мүмкін.

2-кесте. ORDA жобасының технологиялық стегі

Қабат	Қолданылған технология	Жобадағы рөлі
Frontend	HTML, CSS, JavaScript	Интерфейс, навигация, іздеу, фильтр, формалар, көптілділік
Backend	Node.js, Express.js	API маршруттары, бизнес-логика, статикалық файлдарды тарату
Database	MongoDB, Mongoose	Пайдаланушылар, іс-шаралар, залдар, өтінімдер және хабарламаларды сақтау
Security	bcryptjs, JSON Web Token	Құпиясөзді қорғау, сессиясыз авторизация, рөлдік қолжетімділік
Configuration	dotenv, CORS	Қоршаған орта айнымалылары және рұқсат етілген origin баптауы

2 ORDA SMART EVENT SYSTEM ЖҮЙЕСІН ЖОБАЛАУ

2.1 Жүйеге қойылатын талаптар және пайдаланушы сценарийлері

Жүйені жобалау кезінде үш негізгі рөл қарастырылды: тіркелмеген қонақ, тіркелген пайдаланушы және әкімші. Қонақ іс-шаралар мен залдар туралы жалпы ақпаратты қарай алады. Тіркелген пайдаланушы іс-шараға жазылады, залға немесе өз іс-шарасын жариялауға өтінім береді, төлем мәртебесін жаңартады, жеке кабинетте өтінімдерін қарайды және әкімшіге техникалық қолдау хабарламасын жібереді. Әкімші каталогты, өтінімдерді, статистиканы, залдарды және клиент хабарламаларын басқарады.

3-кесте. Пайдаланушы рөлдері және сценарийлері

Рөл	Негізгі әрекеттер	Қолжетімділік шарты
Қонақ	Басты бет, іс-шаралар, залдар, күнтізбе, карта және ақпараттық беттерді қарау	Авторизация қажет емес
Пайдаланушы	Тіркелу, кіру, іс-шараға жазылу, зал брондау, төлем белгілеу, жеке кабинет, қолдау хабары	JWT токени қажет
Әкімші	Іс-шара/зал құру, өңдеу, жою, өтінім бекіту/қайтару, check-in, статистика, support	JWT және admin рөлі қажет

Жүйенің негізгі функционалдық талаптары төмендегідей анықталды:

- пайдаланушыны тіркеу және жүйеге кіргізу;
- бірінші тіркелген пайдаланушыны автоматты түрде әкімші ету;
- іс-шараларды іздеу, категория, формат және тегтер бойынша сүзгілеу;
- іс-шараға қатысушы ретінде жазылу және бос орын санын бақылау;
- конференц-залдар каталогын көрсету және сыйымдылық бойынша сүзу;
- залға өтінім беру кезінде уақыт қақтығысын анықтау;
- әкімшіге өтінім мәртебесін өзгерту және клиентке хабарлама көрсету;
- жарық/қараңғы тақырып пен RU/KZ/EN тілдерін қолдау;
- техникалық қолдау хабарламаларын жіберу және әкімші тарапынан өңдеу.

Талаптарды қалыптастыру кезеңінде жүйенің негізгі акторлары анықталды. Қонақ пайдаланушы ақпаратты қарайды және жүйенің мүмкіндіктерімен танысады. Тіркелген пайдаланушы іс-шараға жазылады, залға өтінім береді, жеке деректерін жаңартады және мәртебелерді бақылайды. Әкімші деректерді басқару, өтінімдерді қарау, статистиканы бақылау және техникалық қолдау хабарламаларын өңдеу міндетін атқарады. Мұндай рөлдік бөлу әр функцияның кімге арналғанын нақтылауға мүмкіндік береді.

Функционалдық талаптар жүйенің не істеуі керектігін сипаттайды. ORDA жағдайында бұған тіркелу, кіру, іс-шара каталогын қарау, фильтрлеу, брондау, залға өтінім беру, admin dashboard, check-in және support модулі кіреді. Ал функционалдық емес талаптар жүйенің қалай жұмыс істеуі керектігін

көрсетеді: жауап беру жылдамдығы, қауіпсіздік, қолжетімділік, кеңейтілу, интерфейсін түсініктілігі және деректердің тұрақты сақталуы.

Пайдаланушы сценарийлерін талдау жобалаудың маңызды бөлігі болды. Мысалы, іс-шараға тіркелу сценарийінде пайдаланушы алдымен каталогтан event таңдайды, содан кейін жүйеге кіреді немесе тіркеледі, кейін серверге booking сұранысы жіберіледі. Сервер eventId арқылы іс-шараны табады, пайдаланушының бұрын жазылған-жазылмағанын тексереді, бос орын санын есептейді және жаңа Booking құжатын жасайды. Бұл сценарий қарапайым батырма басудан басталғанымен, бірнеше ішкі тексерісті қамтиды.

Залға өтінім беру сценарийі күрделілеу. Пайдаланушы залды таңдайды, күн мен уақытты енгізеді, қатысушылар санын, мақсатты, қызметтерді және төлем әдісін көрсетеді. Сервер залдың бар-жоғын, сыйымдылығын, уақыттың дұрыс енгізілгенін және қақтығыстың жоқтығын тексереді. Егер өтінім custom-event түрінде болса, әкімші бекіткеннен кейін ол жеке іс-шара ретінде каталогқа жариялануы мүмкін. Бұл процесс жүйенің брондау мен event management логикусын біріктіретінін көрсетеді.

Әкімші сценарийлері ұйымның ішкі жұмысын қолдайды. Әкімші жаңа іс-шара жариялайды, зал туралы ақпаратты жаңартады, өтінім мәртебесін өзгертеді, қатысушыны check-in арқылы белгілейді, төлем мәртебесін қарайды және support хабарламаларын өңдейді. Әкімші әрекеттері жүйеде қадағаланатын мәртебелер арқылы көрінеді, сондықтан пайдаланушы да жеке кабинеттен шешім нәтижесін бақылай алады.

Талаптарды осылай бөлу жобаның шекарасын нақтылайды. Дипломдық жоба төлем шлюзін толық іске қоспаса да, төлем мәртебесі мен төлем әдісін сақтау арқылы болашақ интеграцияға дайындалады. Email немесе SMS хабарлама әзірше толық іске қосылмаса да, notification логикусы мен support модулі пайдаланушыға кері байланыс беру бағытын көрсетеді. Яғни жүйе қазіргі мүмкіндігімен жұмыс істейді және кеңейту нүктелерін сақтайды.

2.2 Клиент-сервер архитектурасы және деректер ағыны

ORDA веб-жүйесінде клиенттік және серверлік қабаттар нақты бөлінген. Клиенттік бөлік `docs` каталогында орналасқан және браузерде орындалады. Серверлік бөлік `backend/src` каталогында орналасып, Express қосымшасы ретінде API сұраныстарын өңдейді. Негізгі `server.js` файлы backend серверін іске қосады, ал Express статикалық файлдарды таратып, `/api` кеңістігіндегі маршруттарға жауап береді.

Деректер ағыны келесі тәртіппен жүреді: пайдаланушы интерфейсте әрекет жасайды, JavaScript HTTP сұранысын жібереді, Express маршруты тиісті controller функциясын шақырады, controller Mongoose моделі арқылы MongoDB деректер қорымен жұмыс істейді және JSON жауап қайтарады. Клиент жауапқа байланысты интерфейсін жаңартады.

Клиент-сервер архитектурасының басты артықшылығы - жауапкершілікті бөлу. Клиенттік қабат пайдаланушыға интерфейс ұсынады, енгізілген деректерді бастапқы деңгейде тексереді және серверге сұраныс жібереді. Серверлік қабат негізгі бизнес-логиканы орындайды, деректер қорымен жұмыс

істейді және қауіпсіздік тексерістерін жүргізеді. Деректер қоры тұрақты сақтау міндетін атқарады. Бұл бөлініс жүйені түсінуді, тестілеуді және кеңейтуді жеңілдетеді.

ORDA жүйесінде frontend `docs` каталогында орналасқан статикалық файлдардан тұрады. `index.html` бет құрылымын береді, `styles.css` визуалды дизайнды қалыптастырады, ал `app.js` негізгі интерактивті логиканы басқарады. Core модульдерде http helper, session helper, i18n, theme, toast және форматтау функциялары бөлінген. Бұл frontend кодын толық монолит емес, бірнеше жауапкершілік аймағына бөлінген құрылым ретінде қарастыруға мүмкіндік береді.

Backend `backend/src` құрылымында қабаттарға бөлінген. `config` каталогында орта айнымалылары мен MongoDB connection логикасы орналасады. `models` Mongoose схемаларын сипаттайды. `middleware` авторизация, рөл тексеру, валидация және қате өңдеу міндеттерін орындайды. `controllers` бизнес-логиканы қамтиды, ал `routes` HTTP endpoint-терді controller функцияларымен байланыстырады. Мұндай құрылым Express жобалары үшін түсінікті және қолдауға ыңғайлы.

Деректер ағынын мысал арқылы көрсетуге болады. Пайдаланушы login формасын толтырғанда frontend `/login` endpoint-іне POST сұраныс жібереді. Backend email мәнін normalize етеді, пайдаланушыны деректер қорынан іздейді, bcrypt арқылы құпиясөзді салыстырады және JWT токенін қайтарады. Frontend бұл токенді session helper арқылы сақтап, келесі қорғалған сұраныстарға қосады. Осылайша authentication күйі клиентте сақталған белгі мен сервердегі тексерістің үйлесімі арқылы орындалады.

Іс-шаралар каталогын жүктеу кезінде frontend search, category, format немесе free сияқты query параметрлерін API-ге жібереді. Event controller MongoDB query құрып, жарияланған іс-шараларды дата мен уақыт бойынша сұрыптайды. Әр іс-шара үшін booked count және пайдаланушының бұрын тіркелгені туралы белгі есептеледі. Бұл ақпарат frontend-ке қайтып келген соң карточкадағы бос орын, тіркелу батырмасы және мәртебе визуалды түрде жаңартылады.

Қате өңдеу архитектураның маңызды бөлігі болып табылады. Егер сұраныс дұрыс болмаса, controller ApiError арқылы мағыналы статус пен хабарлама қайтарады. Error handler бұл қатені JSON жауапқа айналдырады. Frontend showMessage немесе toast механизмі арқылы пайдаланушыға түсінікті хабарлама көрсетеді. Осылайша техникалық қате ішкі деңгейде өңделіп, интерфейсте басқарылатын кері байланысқа айналады.

Жүйе `/api` namespace арқылы да, кейбір бұрынғы frontend сұраныстарымен үйлесімді болу үшін namespace-сыз да маршруттарды қолдайды. Бұл шешім әзірлеу кезінде пайдалы, себебі frontend кодын бірден толық ауыстырмай, API құрылымын біртіндеп жаңартуға мүмкіндік береді. Дегенмен production деңгейінде endpoint келісімін нақты құжаттап, бір негізгі namespace қолдану жүйені қолдауды жеңілдетеді.

4-кесте. Жоба құрылымындағы негізгі қабаттар

Қабат	Каталог/файл	Міндеті
Static frontend	docs/index.html, docs/styles.css, docs/app.js	Интерфейс құрылымы, стильдер және қолданушы әрекеттері
Core frontend	docs/js/core/*.js	HTTP helper, session, i18n, theme, toast, map, domain helpers
Express app	backend/src/app.js	CORS, JSON parser, static files, health, API router, error handler
Routes	backend/src/routes/*.js	Auth, user, event, hall, booking, admin, support endpoints
Controllers	backend/src/controllers/*.js	Бизнес-логика және жауап форматтары
Models	backend/src/models/*.js	MongoDB коллекцияларының схемалары

2.3 Деректер қоры құрылымы, API логикасы және рөлдік қолжетімділік

MongoDB құжаттық деректер қоры ORDA жүйесіне ыңғайлы, себебі іс-шара, зал, өтінім және пайдаланушы объектілері әртүрлі өрістерден тұрады және уақыт өте кеңейтілуі мүмкін. Mongoose схемалары арқылы әр коллекцияның міндетті өрістері, типтері, enum мәндері және бастапқы мәндері анықталған.

5-кесте. ORDA жүйесіндегі деректер модельдері

Коллекция	Негізгі өрістер	Мақсаты
User	name, email, password, role, phone, organization, city	Пайдаланушы профилі және рөлдік қолжетімділік
Event	title, date, time, category, location, format, price, capacity, status	Іс-шара каталогы және қатысушы тіркеуі
Hall	name, floor, location, capacity, pricePerHour, equipment, status	Конференц-залдар каталогы
Booking	type, userEmail, eventId, hallId, date, time, duration, amount, status	Іс-шараға жазылу, зал брондау және клиент өтінімдері
SupportMessage	userName, userEmail, text, status, readAt	Техникалық қолдау хабарламалары

API логикасында маршруттар екі нұсқада қолжетімді: `/api` namespace арқылы және кейбір ескі frontend сұраныстарымен үйлесімді болу үшін namespace-сыз. Бұл шешім интерфейсті біртіндеп жаңартуға мүмкіндік береді. Қорғалған маршруттарда `auth` middleware, әкімші маршруттарында `adminOnly` middleware қолданылады.

Деректер қоры құрылымында User коллекциясы пайдаланушының негізгі деректерін сақтайды. Мұнда name, email, password, role, phone, organization және city өрістері бар. Email бірегей болуы тиіс, ал role user немесе admin мәндерінің бірін қабылдайды. Бұл модель авторизация мен рөлдік қолжетімділіктің негізін құрайды. Күпиясөздің базаға хэш түрінде сақталуы жеке деректерді қорғаудың маңызды шарты болып табылады.

Event коллекциясы іс-шара туралы толық ақпаратты қамтиды: title, date, endDate, time, category, description, agenda, organizer, location, city, format,

meetingUrl, image, price, currency, featured, tags, capacity және status. Осындай өрістер каталог карточкасын, event detail бөлімін, фильтрлеуді және admin формасын қамтамасыз етеді. Status өрісінің draft, published және closed мәндері іс-шараның өмірлік циклін көрсетуге мүмкіндік береді.

Hall коллекциясы конференц-залдың сипаттамасын сақтайды. Name, floor, location, capacity, pricePerHour, equipment, description, translations, status, image және advantages өрістері зал туралы толық ақпарат беруге арналған. Capacity залға өтінім беру кезінде қатысушылар санын тексеруге қолданылады, ал status available, busy немесе maintenance күйін көрсетуі мүмкін. Equipment және advantages массивтері залдың техникалық мүмкіндіктерін интерфейсте көрсетуге ыңғайлы.

Booking коллекциясы жүйенің ең маңызды байланыс объектілерінің бірі. Ол event тіркелуі, hall өтінімі және custom-event сұранысы сияқты бірнеше типті бір модельде сақтайды. Type өрісі event, hall немесе custom-event мәнін алады. Сонымен қатар userEmail, userName, eventId, hallId, date, time, duration, attendees, services, equipment, amount, paymentStatus, status, checkedIn және adminReason сияқты өрістер процестің толық күйін сипаттайды.

SupportMessage коллекциясы пайдаланушы мен әкімші арасындағы техникалық байланыс арнасын қамтамасыз етеді. Онда userId, userName, userEmail, text, status және readAt өрістері сақталады. Мұндай модель қарапайым көрінгенімен, жүйені қолдау сапасын арттырады: пайдаланушы сұрақ қалдырады, әкімші оны оқиды, мәртебесін белгілейді және қажет болған жағдайда жауап беру процесін ұйымдастырады.

API логикасында әр модельге сәйкес controller жауапкершілігі бөлінген. Auth controller тіркелу мен кіруді өңдейді, Event controller каталог пен event CRUD операцияларын басқарады, Hall controller залдарды өңдейді, Booking controller пайдаланушы өтінімдерін жасайды және қайтарады, Admin controller жалпы басқару функцияларын орындайды, Support controller хабарламалармен жұмыс істейді. Бұл бөлініс кодты іздеуді және өзгертуді жеңілдетеді.

Деректердің тұтастығын сақтау үшін сервер деңгейінде бірнеше тексеріс қолданылады. Event жасау кезінде міндетті өрістер тексеріледі, зал брондау кезінде уақыт қақтығысы анықталады, іс-шараға тіркелуде қайталанған booking пен capacity шектеуі бақыланады. Бұл тексерістерді тек frontend формасына сеніп қою жеткіліксіз, себебі API-ге сұранысты басқа құрал арқылы да жіберуге болады. Сондықтан негізгі валидация backend деңгейінде орындалуы тиіс.

MongoDB Atlas сияқты бұлттық база қолданылған жағдайда жүйе локалды компьютерден тәуелсіз сақтауға ие болады. Бұл демонстрация кезінде де, болашақ production орналастыруда да ыңғайлы. Дегенмен бұлттық базаға IP whitelist, connection string құпиялығы, '.env' файлын қорғау және backup стратегиясы қажет. Осы талаптар дипломдық жобада қауіпсіздік пен масштабталудың практикалық аспектілерін түсіндіруге мүмкіндік береді.

База құрылымын болашақта оңтайландыру үшін индекстер мәселесі де қарастырылуы мүмкін. Мысалы, Event коллекциясында date, status, category және format өрістеріне сұраныс жиі түседі, ал Booking коллекциясында userEmail, eventId, hallId, date және status бойынша іздеу маңызды. Индекстер

дұрыс қойылса, деректер саны артқан кезде каталог жүктелуі, жеке кабинет сұраныстары және admin dashboard статистикасы жылдамырақ орындалады. Алайда индекстерді шамадан тыс көбейту жазу операцияларын баяулатуы мүмкін, сондықтан олар нақты сұраныс үлгісіне сүйеніп таңдалуы тиіс.

Деректер қорын жобалауда резервтік көшіру және қалпына келтіру жоспары да практикалық мәнге ие. Іс-шаралар, өтінімдер және қатысушылар тізімі ұйым үшін маңызды ақпарат болып табылады, сондықтан база бұзылған немесе кездейсоқ өшірілген жағдайда деректерді қайта қалпына келтіру мүмкіндігі болуы керек. MongoDB Atlas snapshot, backup және monitoring құралдарын ұсына алады, ал шағын орналастыруда экспорт файлдарын кезеңдік сақтау бастапқы қорғаныс шарасы ретінде қарастырылуы мүмкін.

6-кесте. API маршруттарының қысқаша сипаттамасы

Модуль	Endpoint үлгілері	Қорғаныс
Auth	POST /api/register, POST /api/login	Ашық, бірақ валидация бар
Profile	GET /api/me, PUT /api/me	auth
Events	GET /api/events, POST/PUT/DELETE /api/events/:id	Оқу ашық/optionalAuth, өзгерту adminOnly
Bookings	POST /api/book, GET /api/my-bookings, DELETE /api/my-bookings/:id	auth
Halls	GET /api/halls, POST/PUT/DELETE /api/halls/:id	Оқу ашық, өзгерту adminOnly
Admin	GET /api/bookings, GET /api/stats, PATCH /api/bookings/:id	auth + adminOnly
Support	POST /api/support-messages, GET/PATCH/DELETE /api/support-messages	Жібепу auth, басқару adminOnly

2.4 Интерфейсті жобалау, көптілділік және визуалдық стиль

Интерфейс SaaS стиліндегі жұмыс панелі ретінде жобаланған. Бетте жоғарғы header, жаһандық іздеу, тіл таңдауы, тақырып ауыстырғыш, account аймағы және бүйірлік навигация бар. Навигацияда басты бет, іс-шаралар, залдар, күнтізбе, карта, профиль, өтінім және ақпарат бөлімдері орналасады. Әкімші кірген кезде қосымша admin бөлімі ашылады.

Көптілділік `docs/js/core/i18n.js` файлы арқылы ұйымдастырылған. Интерфейстегі мәтіндер `data-i18n` атрибуттары арқылы байланыстырылады, ал пайдаланушы RU, KZ немесе EN тілін таңдай алады. Бұл шешім бір HTML құрылымын сақтай отырып, мәтіндік қабатты бөлек басқаруға мүмкіндік береді.

Интерфейсті жобалауда негізгі мақсат - пайдаланушыны ақпаратпен шамадан тыс жүктемей, жиі орындалатын әрекеттерді тез табуға мүмкіндік беру. ORDA жүйесінде dashboard логикасы қолданылған: бүйірлік навигация негізгі бөлімдерді тұрақты көрсетеді, ал орталық аймақта таңдалған view мазмұны ауысады. Бұл тәсіл жеке беттер арасында қайта жүктеусіз ауысуға және қолданушыны бір жұмыс кеңістігінде ұстауға көмектеседі.

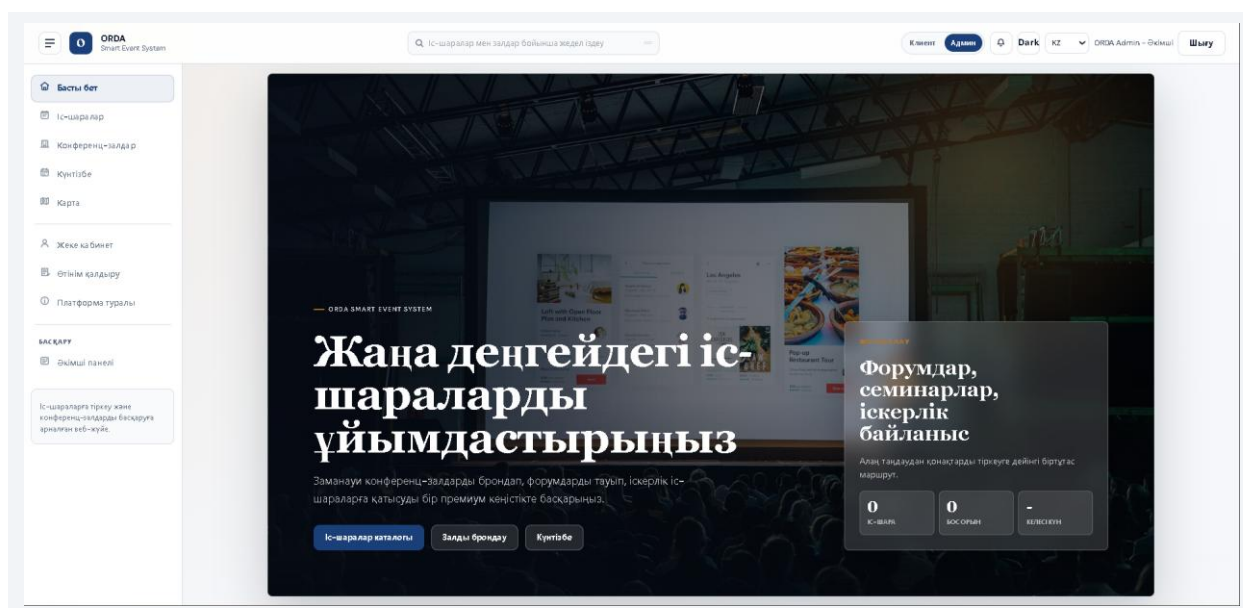
Визуалдық стильде SaaS панельдеріне тән қағидалар байқалады: карточкалар, статус белгілері, фильтрлер, форма блоктары, статистикалық көрсеткіштер және түс арқылы ерекшелену. Мұндай стиль іс-шара жүйесіне сәйкес келеді, себебі пайдаланушы бірден бірнеше объектіні салыстырады: events, halls, bookings, calendar entries және notifications. Әр объектінің карточка немесе жол түрінде көрсетілуі ақпаратты сканерлеуді жеңілдетеді.

Адаптивті дизайн кезінде мазмұнның иерархиясын сақтау қажет. Мобильді экранда бүйірлік навигация, карточка торы және формалар бір бағанға түсуі мүмкін, бірақ негізгі әрекеттер жоғалмауы тиіс. Сондықтан батырмалар жеткілікті үлкен болуы, форма өрістері оқылатын болуы, ал мәтіндер экраннан шықпауы керек. Бұл талаптар CSS деңгейінде breakpoints, flexible layout және spacing жүйесі арқылы шешіледі.

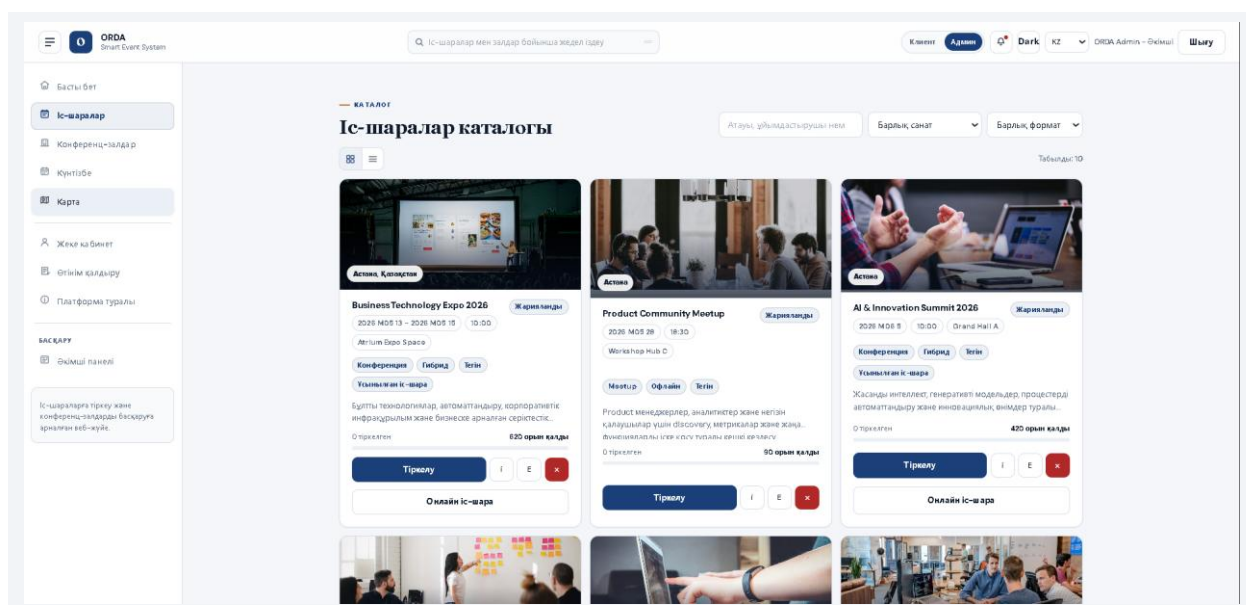
Жарық және қараңғы тақырып пайдаланушының жұмыс жағдайына бейімделуге мүмкіндік береді. Қараңғы тақырып кешкі уақытта немесе проекторда көрсету кезінде ыңғайлы болуы мүмкін, ал жарық тақырып күнделікті кеңсе жұмысында оқуға жеңіл. Тақырып ауыстыру тек түсті өзгерту емес: контраст, мәтіннің оқылуы, border және фон реңктерінің үйлесімі сақталуы тиіс. ORDA жобасында theme helper бұл күйді басқаруға арналған.

Көптілді интерфейсте терминдердің тұрақтылығы маңызды. Бір тілде 'booking', екінші тілде 'өтінім', үшінші тілде 'заявка' деп берілген ұғымдар бір функцияны білдіруі керек. Егер терминдер әр бөлімде әртүрлі қолданылса, пайдаланушы шатасуы мүмкін. Сондықтан i18n сөздіктерін бөлек файлда сақтау тек техникалық шешім ғана емес, интерфейс мәтіндерінің сапасын басқару құралы болып табылады.

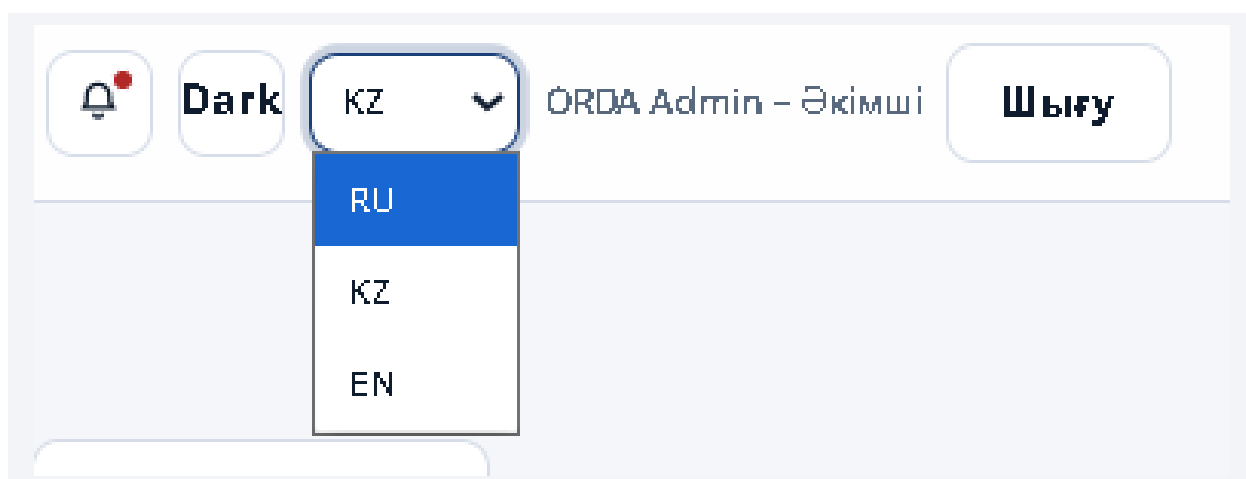
UX/UI шешімдерінің тиімділігі жүйенің қабылдануына тікелей әсер етеді. Тіпті backend дұрыс жұмыс істесе де, пайдаланушы интерфейсті түсінбесе, жүйе практикалық тұрғыдан әлсіз болады. Сол себепті дипломдық жұмыста интерфейс функционалдық модульдердің көрінісі ретінде ғана емес, пайдаланушы мен ақпараттық жүйе арасындағы негізгі байланыс арнасы ретінде қарастырылды.



1-сурет. ORDA жүйесінің басты беті



2-сурет. Іс-шаралар каталогы және сүзгілер



3-сурет. Көптілділік және тақырып ауыстыру

3 ЖҮЙЕНІ ІСКЕ АСЫРУ, ТЕКСЕРУ ЖӘНЕ ТИІМДІЛІГІН БАҒАЛАУ

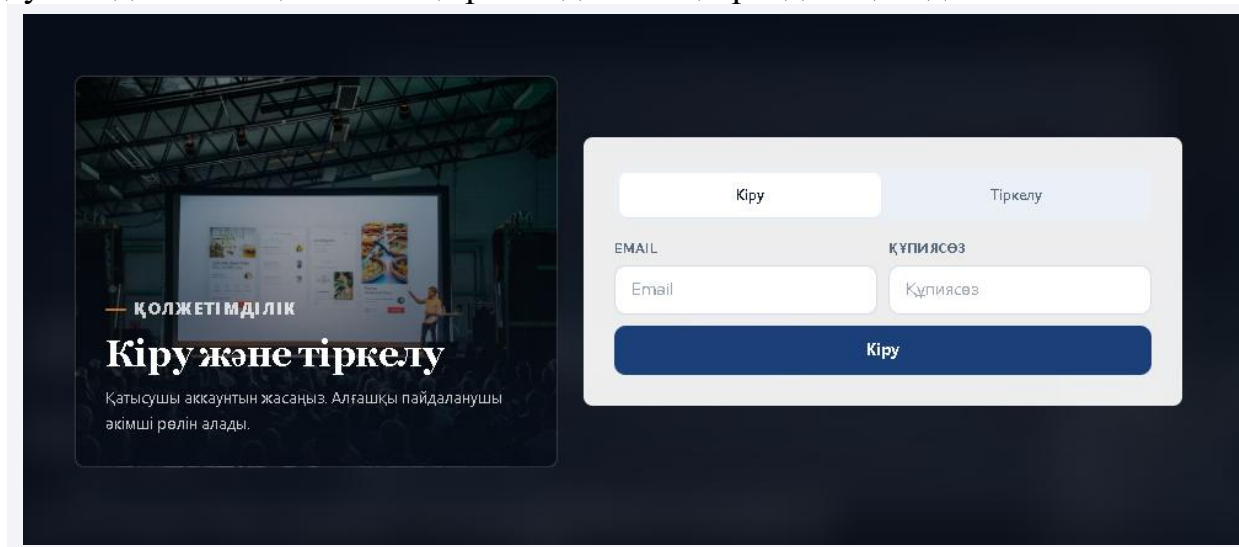
3.1 Клиенттік бөліктің негізгі модульдерін жүзеге асыру

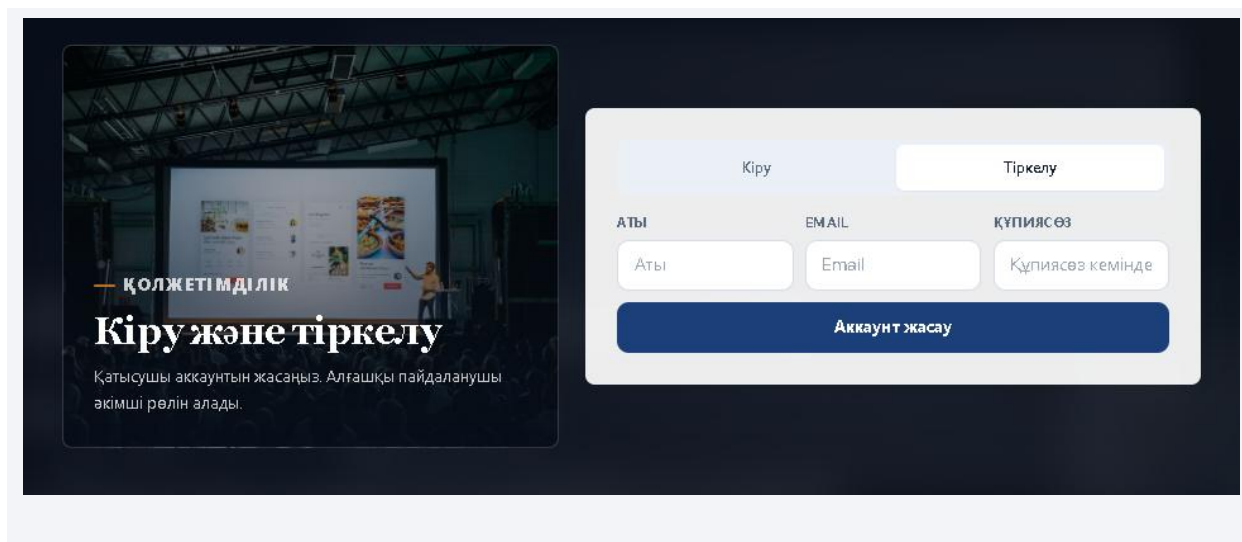
Клиенттік бөліктің негізгі мақсаты - пайдаланушыға іс-шара немесе зал туралы ақпаратты тез табуға, өтінім жіберуге және мәртебесін бақылауға мүмкіндік беру. `docs/index.html` файлында барлық негізгі view бөлімдері орналасқан, ал `docs/app.js` пайдаланушы әрекеттерін, деректерді жүктеуді, формаларды жіберуді және интерфейсті жаңартуды басқарады.

Авторизация модулі login және register формаларынан тұрады. Пайдаланушы email және құпиясөз енгізгеннен кейін сервер JWT токенін қайтарады. Токен браузердегі session helper арқылы сақталып, кейінгі қорғалған сұраныстардың `Authorization: Bearer <token>` header-іне қосылады.

Frontend деңгейінде авторизация формалары пайдаланушының жүйемен алғашқы белсенді байланысын құрайды. Register формасында аты, email және құпиясөз енгізіледі, ал login формасы бұрын тіркелген пайдаланушыны тануға арналған. Email мәнін normalize ету және құпиясөздің минималды ұзындығын тексеру қате деректер санын азайтады. Сәтті кіргеннен кейін интерфейс пайдаланушы рөліне қарай бейімделеді: қарапайым user мен admin әртүрлі бөлімдерді көреді.

Session helper токенді сақтау және оны кейінгі сұраныстарға қосу міндетін атқарады. Бұл механизм пайдаланушы әр әрекет сайын қайта логин жасамауы үшін қажет. Сонымен қатар мерзімі өткен немесе жарамсыз токен анықталса, session reset логикасы қолданушыны қайта кіруге бағыттайды. Мұндай шешім қауіпсіздік пен ыңғайлылық арасындағы теңгерімді сақтайды.





Іс-шара модулінде пайдаланушы жарияланған events тізімін көреді. Карточкада атауы, күні, уақыты, орны, форматы, бағасы, бос орын саны және тіркелу батырмасы көрсетіледі. Жүйе пайдаланушының бұрын тіркелгенін анықтап, 'isBookedByMe' белгісі арқылы интерфейсін өзгертеді.

Іс-шара модулінде деректерді көрсету ғана емес, оны іздеу және сүзу маңызды. Search өрісі атау, сипаттама, орын, ұйымдастырушы немесе тегтер бойынша ақпарат табуға көмектеседі. Category және format филтрлері пайдаланушыға қажетті мазмұнды тез шектеуге мүмкіндік береді. Featured белгісі маңызды іс-шараларды бөлек көрсетуге, ал free филтрі ақысыз events тізімін алуға қолданылады.

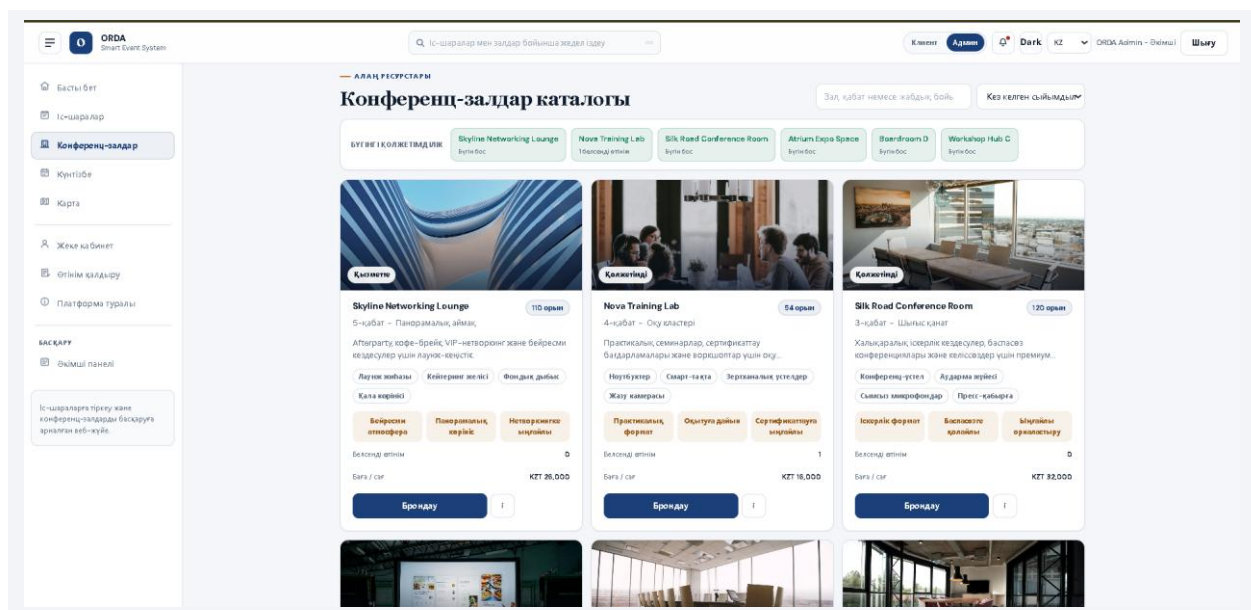
Event карточкасының құрылымы шешім қабылдауға қажетті қысқа ақпаратты біріктіреді. Күні мен уақыты қатысу мүмкіндігін, location зал немесе қала контекстін, price қаржылық шартты, capacity және booked count бос орын санын көрсетеді. Егер пайдаланушы бұрын тіркелген болса, интерфейс сол күйді көрсетуі керек. Бұл қайталанған әрекетті азайтады және пайдаланушыға жүйенің оның тарихын есте сақтайтынын байқатады.

5-сурет. Іс-шара карточкасы және тіркелу әрекеті

Залдар модулі конференц-залдар каталогын көрсетеді. Пайдаланушы залды сыйымдылық бойынша сүзеді, сипаттамасын, бағасын, жабдығын және бос уақытын қарайды. Залға өтінім беру кезінде күн, уақыт, ұзақтық, қатысушылар саны, мақсат, қосымша қызметтер және төлем әдісі енгізіледі.

Зал модулі кеңістікті ресурс ретінде басқаруға арналған. Әр залдың сыйымдылығы, қабаты, орналасуы, сағаттық бағасы, жабдықтары және сипаттамасы болады. Бұл деректер клиентке дұрыс зал таңдауға көмектеседі, ал әкімші үшін зал ресурстарын құрылымдауға мүмкіндік береді. Егер зал maintenance күйінде болса немесе оған белсенді өтінімдер болса, оны жою немесе қайта брондау қосымша тексерісті қажет етеді.

Брондау формасында енгізілетін деректер кейін бірнеше есептеуге қатысады. Duration мәні startTime арқылы endTime есептеуге көмектеседі, attendees зал capacity мәнімен салыстырылады, services және equipment қосымша қажеттіліктерді көрсетеді, paymentMethod төлем сценарийін сипаттайды. Осындай деректердің бір модельге жиналуы әкімшіге өтінімді толық бағалауға мүмкіндік береді.



6-сурет. Конференц-залдар каталогы

Брондау: Nova Training Lab

4-қабат – Оқу кластері – 54 адамға дейін – KZT 16,000 / сағ

КҮНІ

ДД.ММ.ГГГГ

БАСТАЛУ УАҚЫТЫ

09:00

ҰЗАҚТЫҒЫ (САҒ)

2

ҚАТЫСУШЫЛАР САНЫ

30

ЖАБДЫҚ

☐ ПРОЕКТОР

☐ МИКРОФОНДАР

☐ ТРАНСЛЯЦИЯ

☐ КЕЙТЕРИНГ

БРОНДАУ МАҚСАТЫ

Мысалы: стратегиялық сессия, салалық семинар

Өтінім жіберу

Бас тарту

7-сурет. Залды брондау өтінімі

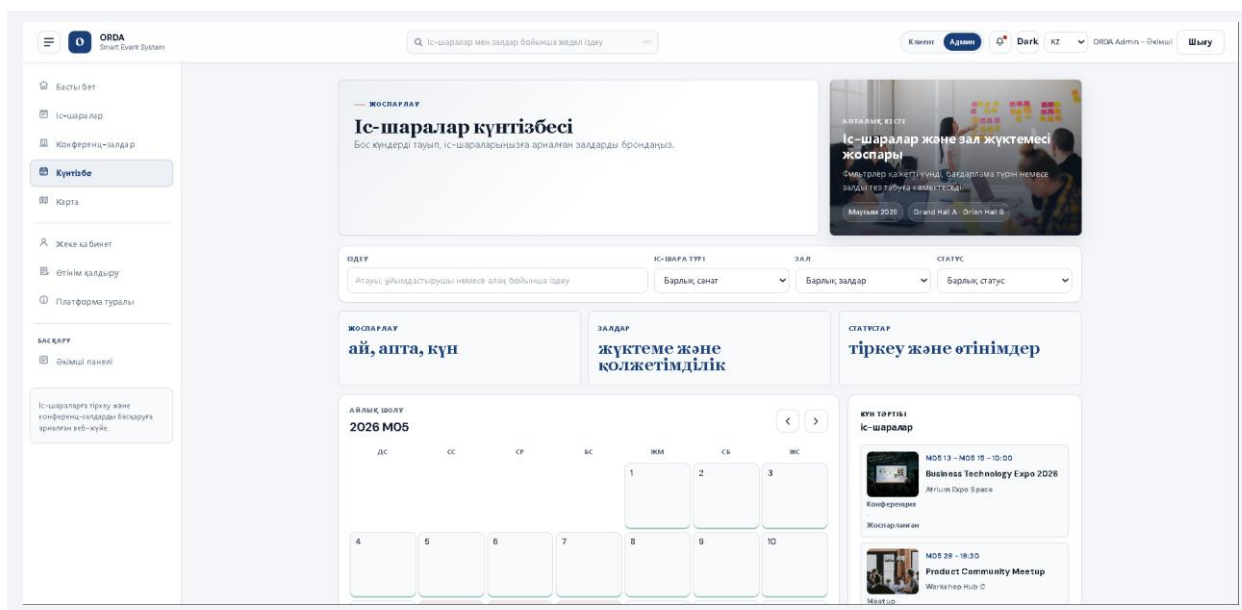
Күнтізбе және floorplan модульдері ұйымдастырушыға жүктемені визуалды түрде көруге көмектеседі. Күнтізбеде іс-шаралар, зал брондаулары және клиент өтінімдері бір күнге біріктіріліп көрсетіледі. Карта немесе floorplan бөлімі кеңістікті таңдауды түсінікті етеді.

Күнтізбе модулі уақытқа байланысты ақпаратты визуалды ұйымдастыру құралы болып табылады. Тізім түріндегі өтінімдер көп болған кезде әкімші

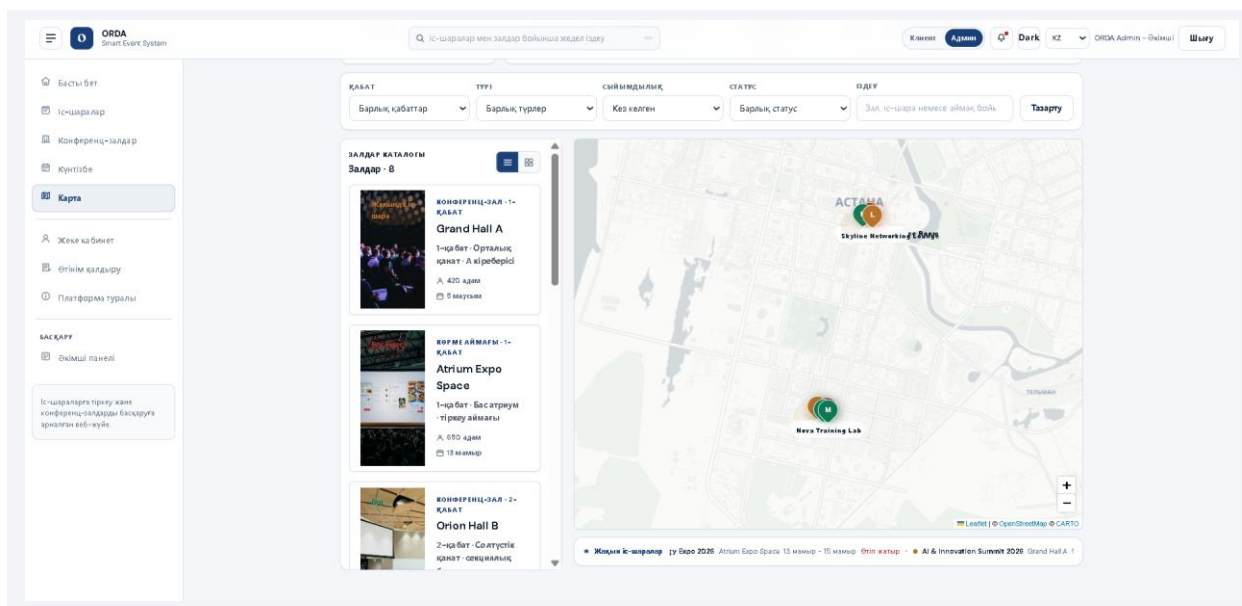
28

нақты күндегі жүктемені бірден байқамауы мүмкін, ал calendar view оқиғаларды уақыт осінде көруге мүмкіндік береді. Бұл зал қақтығысын алдын ала байқауға, күнді жоспарлауға және event пен hall booking арасындағы байланысты түсінуге көмектеседі.

Floorplan немесе карта модулі кеңістікті мәтіндік тізімнен гөрі түсініктірек көрсетеді. Пайдаланушы залдың қай жерде орналасқанын, сыйымдылығын және ағымдағы күйін визуалды қабылдай алады. Егер жүйе болашақта нақты ғимарат жоспарына немесе интерактивті картаға толық қосылса, зал таңдау процесі одан әрі жеңілдейді. Бұл әсіресе үлкен конференц-орталықтар мен бірнеше қабатты оқу ғимараттары үшін пайдалы.



8-сурет. Күнтізбе модулі



9-сурет. Floorplan немесе зал картасы

Жеке кабинетте пайдаланушы өзінің профилін жаңарта алады, іс-шараға жазылуларын, зал өтінімдерін және техникалық қолдау хабарламаларын

карайды. Бұл бөлім клиент пен әкімші арасындағы байланыс арнасын толықтырады.

Жеке кабинет пайдаланушы үшін жүйедегі негізгі бақылау орталығы қызметін атқарады. Мұнда профиль деректері, іс-шараға тіркелулер, зал өтінімдері және хабарламалар бір жерде көрсетіледі. Бұл пайдаланушыға бұрын жіберген өтінімдерін іздеп жүрмеуге, мәртебені өз бетінше тексеруге және қажет болса тіркелуді тоқтатуға мүмкіндік береді. Жеке кабинет жүйенің ашықтығын арттырады.

Профиль деректерін жаңарту ұйыммен байланысты нақтылауға көмектеседі. Phone, organization және city өрістері қатысушыны немесе клиентті кейін тануға, әкімшіге қосымша байланыс жасауға және аудитория құрылымын талдауға мүмкіндік береді. Мұндай деректерді жинау кезінде артық ақпарат сұрамау және тек жүйе жұмысына қажет мәліметтермен шектелу жеке деректерді қорғау қағидасына сәйкес келеді.

10-сурет. Пайдаланушының жеке кабинеті

3.2 Серверлік бөлікті, брондау логикусын және әкімші модулін іске асыру

Серверлік бөлік Express қосымшасы ретінде құрылған. 'backend/src/app.js' файлында CORS баптауы, JSON body parser, статикалық frontend тарату, '/health' маршруты, API router және error handler қосылған. Мұндай құрылым серверді бір жағынан web app ретінде, екінші жағынан REST API ретінде қолдануға мүмкіндік береді.

Авторизация controller-і email мекенжайын normalize етеді, құпиясөз ұзындығын тексереді, қайталанған email-ді болдырмайды және бірінші пайдаланушыны admin рөлімен тіркейді. Бұл дипломдық демонстрация үшін ыңғайлы, себебі бос базада жеке admin құрудың қосымша скрипті қажет емес.

Брондау логикусында екі маңызды тексеріс бар: біріншісі - пайдаланушының бір іс-шараға екі рет тіркелмеуі, екіншісі - орын санының capacity мәнінен аспауы. Зал брондау кезінде 'findScheduleConflicts' функциясы белгілі бір зал, күн және уақыт аралығы бойынша бұрынғы өтінімдер мен іс-

шараларды салыстырады. Егер уақыт аралықтары қиылысса, сервер қате қайтарады.

Әкімші модулі жүйені басқарудың орталық бөлігі болып табылады. Әкімші барлық өтінімдерді көреді, мәртебесін `approved`, `rejected`, `cancelled`, `review` сияқты мәндерге өзгерте алады, check-in жасайды, статистиканы қарайды және техникалық қолдау хабарламаларын оқылған деп белгілейді. Егер custom-event өтінімі бекітілсе, жүйе оны Event коллекциясына жарияланған іс-шара ретінде қоса алады.

Серверлік бөлікте controller логикасы нақты бизнес-әрекеттерге бағытталған. Auth controller тіркелу және кіру кезінде пайдаланушыны тексереді; Event controller іс-шараны жасау, жаңарту, жою және тізімдеу міндетін орындайды; Hall controller залдарды басқарады; Booking controller пайдаланушы өтінімдерін жасайды және жеке өтінімдерді қайтарады; Admin controller жалпы бақылау, статистика, мәртебе өзгерту және check-in функцияларын қамтамасыз етеді.

Брондау логикасындағы `findScheduleConflicts` функциясы уақыт аралықтарының қиылысуын тексереді. Ол start және end минуттарын есептеп, жаңа өтінімнің бұрынғы booking немесе event уақыттарымен қабаттасуын анықтайды. Қақтығыс шарты `startA < endB && endA > startB` түрінде беріледі. Бұл формула екі интервалдың ортақ бөлігі бар-жоғын анықтаудың кең таралған және түсінікті тәсілі болып табылады.

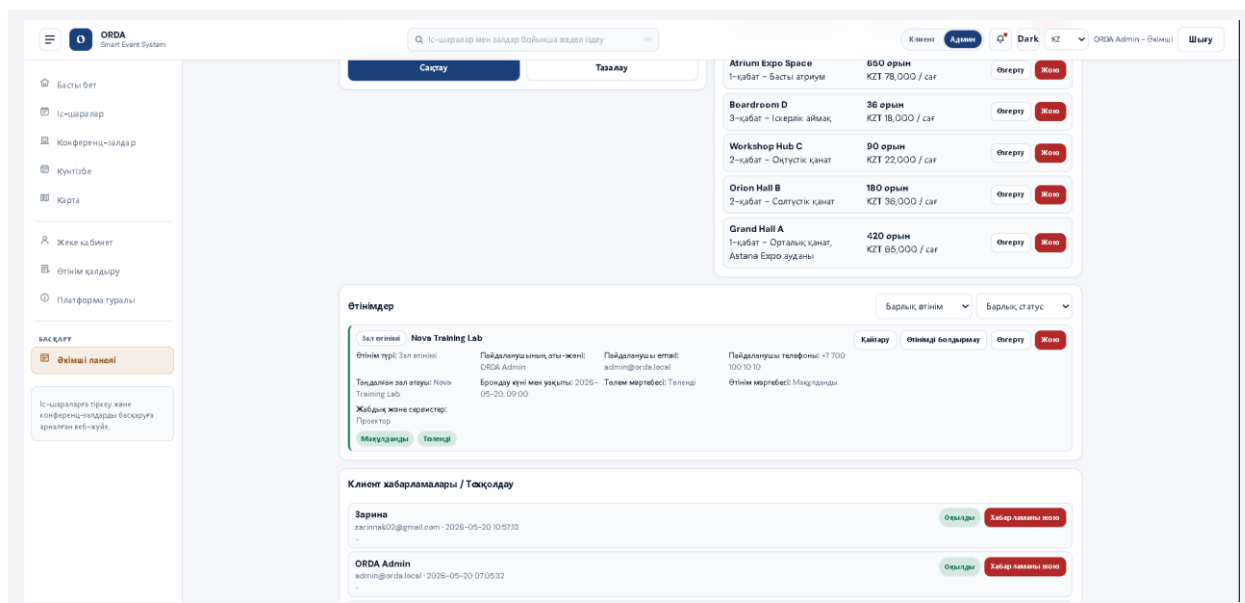
Зал атауларының бірнеше тілде берілуі қақтығысты тексеруді күрделендіреді. Бір зал қазақша, орысша немесе ағылшынша атаумен көрсетілуі мүмкін, сондықтан жүйе hallId ғана емес, hallName, location және translations өрістерін де салыстырады. Бұл көптілді интерфейстің backend логикасына да әсер ететінін көрсетеді: тіл тек мәтін ауыстыру емес, деректерді сәйкестендіру мәселесімен де байланысты.

Admin controller өтінім мәртебесін өзгерткенде бірнеше қосымша әрекет орындай алады. Егер custom-event өтінімі бекітілсе, жүйе оны Event коллекциясына жарияланған іс-шара ретінде қоса алады. Егер өтінім қабылданбаса немесе cancelled болса, төлем мәртебесін refund логикасына байланысты жаңартуға болады. Мұндай байланыстар admin әрекетінің деректер базасында бірнеше объектіге әсер етуі мүмкін екенін көрсетеді.

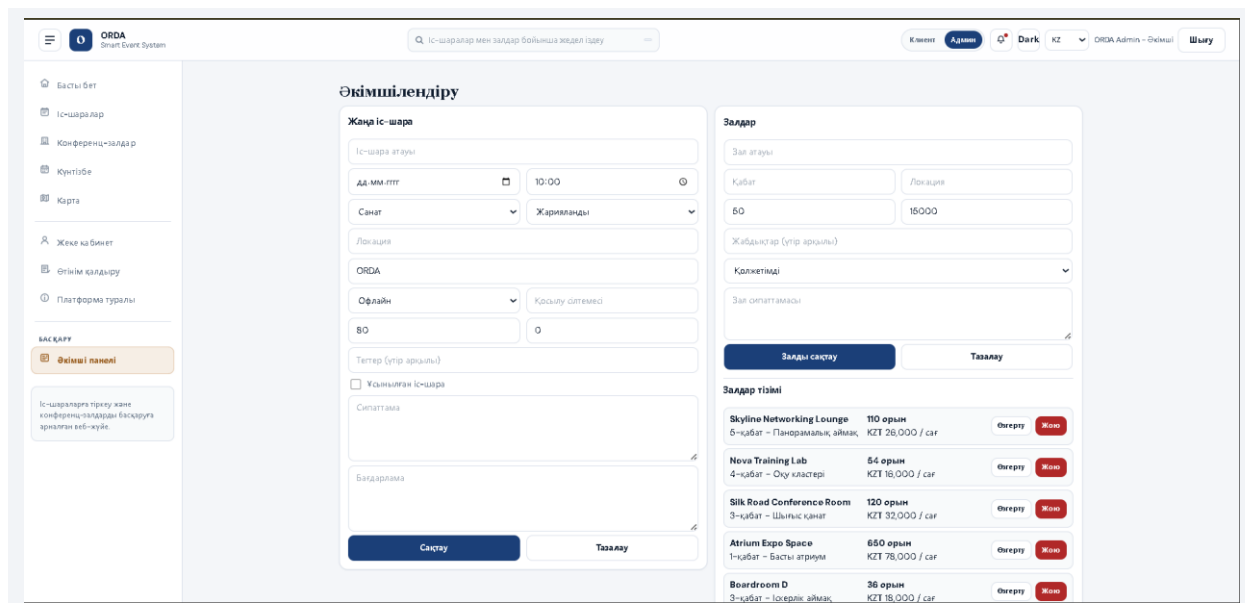
Check-in функциясы іс-шара күні қатысушының нақты келгенін белгілеуге арналған. Бұл функция event management жүйелерінде жиі қолданылады, себебі тіркелген қатысушылардың барлығы іс-шараға келе бермейді. CheckedIn және checkedInAt өрістері кейін статистика жасауға, қатысу көрсеткішін бағалауға және ұйымдастыру сапасын талдауға мүмкіндік береді.

Error handler серверлік архитектураның соңғы қорғаныс деңгейі ретінде жұмыс істейді. Егер controller ішінде қате шықса, ол қолданушыға түсініксіз stack trace ретінде емес, статус коды мен хабарламасы бар JSON жауап ретінде қайтарылады. Бұл frontend-ке қателерді бірдей форматта өңдеуге мүмкіндік береді және жүйенің тұрақтылығын арттырады.

Backend орналастыру тұрғысынан да икемді. Root `server.js` негізгі іске қосу нүктесі ретінде backend серверін шақырады, ал Express қосымшасы frontend статикалық файлдарын да тарата алады. Осылайша жоба бір портта толық веб-қосымша ретінде жұмыс істейді. Кейін production деңгейінде frontend пен backend бөлек орналастырылса да, API логикасы сақталады.



11-сурет. Әкімші панелінің статистикасы



12-сурет. Әкімшінің іс-шара құру/өңдеу формасы

3.3 Тестілеу нәтижелері және анықталған тәуекелдерді талдау

Жүйені тексеру бірнеше деңгейде жүргізілді. Бірінші деңгейде JavaScript файлдарының синтаксисі `npm test` командасы арқылы тексеріледі. Бұл команда root `server.js`, backend сервер файлдары және frontend core модульдері үшін `node --check` орындайды. Екінші деңгейде негізгі пайдаланушы

сценарийлері қолмен тексеріледі: тіркелу, кіру, іс-шараға жазылу, залға өтінім беру, әкімші арқылы мәртебені өзгерту және жеке кабинетте нәтижені көру.

7-кесте. ORDA жүйесін функционалдық тестілеу нәтижелері

№	Тексеру нүктесі	Әрекет	Күтілетін нәтиже	Нәтиже
1	Серверді іске қосу	node server.js	Сайт localhost:3000 арқылы ашылады	Өтті
2	Health endpoint	GET /health	status ok және database мәртебесі қайтады	Өтті
3	Тіркелу	name, email, password енгізу	Пайдаланушы құрылады, бірінші user admin болады	Өтті
4	Логин	email/password енгізу	JWT токени және user деректері қайтады	Өтті
5	Іс-шаралар	GET /api/events және филтрлер	Каталог сұрыпталып, іздеу жұмыс істейді	Өтті
6	Іс-шараға тіркелу	POST /api/book	Booking құрылады, бос орын саны азаяды	Өтті
7	Зал брондау	POST /api/hall-bookings	Өтінім new мәртебесімен сақталады	Өтті
8	Уақыт қақтығысы	Бір залға бір уақытты қайта таңдау	Сервер қате қайтарады	Өтті
9	Әкімші мәртебесі	PATCH /api/bookings/:id	approved/rejected мәртебесі сақталады	Өтті
10	Техникалық қолдау	POST /api/support-messages	Хабарлама admin панельде көрінеді	Өтті

Тестілеу барысында негізгі тәуекелдер де анықталды. Біріншісі - MongoDB Atlas қолжетімсіз болса, деректер жүктелмейді. Бұл жағдайда `/health` endpoint арқылы database мәртебесін тексеру және `.env` параметрлерін қарау қажет. Екіншісі - JWT мерзімі аяқталса, пайдаланушы қайта кіруі керек. Үшіншісі - әкімші емес пайдаланушы admin маршрутқа сұраныс жіберсе, сервер оны тоқтатуы тиіс. Төртіншісі - уақыт қақтығысын тексеру барлық зал атаулары мен аудармаларында дұрыс жұмыс істеуі керек.

Жалпы тест нәтижелері жүйенің негізгі дипломдық мақсаттарын орындайтынын көрсетеді. Интерфейс пен API арасындағы байланыс бар, қорғалған маршруттар жұмыс істейді, өтінім мәртебесі сақталады және әкімші бақылауы қамтамасыз етіледі.

Тестілеу кезінде тек сәтті сценарийлерді емес, қате сценарийлерді де тексеру маңызды. Мысалы, бос email арқылы тіркелу, қысқа құпиясөз енгізу, жарамсыз токенмен protected route ашу, бір іс-шараға қайта жазылу, сарасуы толған event-ке тіркелу және бос емес залға өтінім беру сияқты жағдайлар қарастырылуы тиіс. Мұндай тексерістер жүйенің тұрақтылығын арттырады.

Қауіпсіздік тестілеуі бірнеше бағытты қамтиды. Біріншіден, парольдің ашық мәтінде сақталмауы тексеріледі. Екіншіден, JWT жоқ немесе жарамсыз болған кезде сервер 401 қате қайтаруы керек. Үшіншіден, қарапайым пайдаланушы admin маршрутқа кірсе, 403 деңгейіндегі тыйым орындалуы тиіс.

Төртіншіден, CORS баптауы production режимінде рұқсат етілмеген origin-дерге шектеу қоя алуы керек.

Деректер қоры тәуекелдері де бөлек бағаланды. MongoDB Atlas қолжетімсіз болса, events, halls және bookings жүктелмейді, сондықтан health endpoint арқылы database мәртебесін тексеру маңызды. Connection string пен JWT_SECRET сияқты құпия мәндер `.env` файлында сақталуы керек және репозиторийге жарияланбауы тиіс. Бұл шағын дипломдық жоба үшін де ақпараттық қауіпсіздік мәдениетін қалыптастырады.

Frontend пен backend арасындағы интеграцияны тексеру кезінде API жауаптарының интерфейсте дұрыс көрінуі қаралады. Мысалы, event тізімі келгенде карточкалар жаңаруы, booking жасалған соң жеке кабинетте жаңа жазба пайда болуы, admin мәртебені өзгерткенде пайдаланушыға notification көрінуі қажет. Егер backend дұрыс жауап берсе, бірақ frontend оны көрсетпесе, пайдаланушы үшін жүйе толық жұмыс істемейді.

Өнімділік тұрғысынан дипломдық жоба шағын және орта көлемдегі деректермен жұмыс істеуге бағытталған. Дегенмен болашақта events және bookings саны көбейсе, MongoDB query тиімділігі, индекстер, pagination және кэштеу маңызды болады. Қазіргі архитектура бұл бағыттарға дайын, себебі API филтрлері мен деректер модельдері бөлек ұйымдастырылған.

Тестілеу нәтижелерін талдау жүйені әрі қарай жетілдіру бағыттарын көрсетті. Автоматты unit және integration тесттер қосу, API contract құжаттамасын кеңейту, browser-based end-to-end тесттер енгізу, accessibility audit жүргізу және қауіпсіздік сканерін пайдалану production деңгейіне жақындау үшін маңызды. Осылайша тестілеу тек қате табу емес, жүйенің даму жоспарын нақтылау құралы ретінде қарастырылды.

3.4 Жобаның практикалық және экономикалық тиімділігі

Жобаның практикалық тиімділігі ұйымдастырушының уақытын үнемдеумен және деректерді бір ортаға жинаумен байланысты. Бұрын қатысушы тізімі, зал кестесі, төлем белгісі және клиент хабарламалары бөлек құралдарда сақталуы мүмкін еді. ORDA жүйесінде бұл ақпарат бір деректер қорында орналасады және интерфейс арқылы тез табылады.

Экономикалық тұрғыдан жүйе дайын коммерциялық платформаны толық алмастыруды мақсат етпейді, бірақ оқу орны немесе шағын конференц-орталық үшін бастапқы автоматтандыру құралы бола алады. Жоба ашық технологиялар негізінде жасалғандықтан, лицензиялық шығын азаяды, ал серверді Render, VPS немесе ішкі инфрақұрылымда орналастыруға болады.

Жүйенің кеңейту мүмкіндігі де маңызды. Болашақта QR check-in, нақты төлем жүйесімен интеграция, email/SMS хабарламалары, аналитикалық графиктер, экспорт, push-хабарлама, залдардың 3D картасы және role-based permission жүйесін тереңдету сияқты мүмкіндіктер қосуға болады.

Практикалық тұрғыдан ORDA жүйесі ұйымдастырушының бірнеше күнделікті міндетін қысқартады. Қатысушы тізімін қолмен жинау, залдың бос уақытын бөлек тексеру, өтінім мәртебесін мессенджер арқылы түсіндіру және статистиканы қолмен есептеу орнына жүйе бұл ақпаратты бір ортада сақтайды.

Нәтижесінде әкімші уақыты үнемделеді, ал пайдаланушы өз өтінімінің күйін өз бетінше бақылай алады.

Экономикалық тиімділік тек лицензиялық шығынды азайтумен шектелмейді. Жүйе қызмет көрсету сапасын арттыру арқылы да пайда береді: қателер азаяды, өтінім жоғалмайды, зал қақтығысы алдын ала анықталады, қатысушылар саны нақты бақыланады. Мұндай нәтижелер ұйымның беделіне әсер етеді, себебі қатысушы тіркелу және келу процесін ыңғайлы деп қабылдайды.

Жоба ашық веб-технологияларға сүйенгендіктен, оны қолдау үшін сирек немесе жабық платформа қажет емес. HTML, CSS, JavaScript, Node.js, Express.js және MongoDB кең таралған технологиялар қатарына жатады, олардың құжаттамасы мол және маман табу салыстырмалы түрде жеңіл. Бұл жүйенің ұзақ мерзімді қолдау мүмкіндігін арттырады.

Масштабталу бірнеше деңгейде қарастырылуы мүмкін. Қолданушылар саны артса, backend серверін қуаттырақ ортаға көшіру немесе бірнеше instance арқылы іске қосу қажет болуы мүмкін. Деректер көлемі өссе, MongoDB Atlas кластерін кеңейту, индекстер құру және сұраныстарды оңтайландыру қолданылады. Frontend статикалық файлдарын CDN немесе жеке hosting арқылы тарату жүктемені азайтады.

Перспективалық даму бағыттарының бірі - нақты төлем жүйелерімен интеграция. Қазіргі жүйеде paymentStatus және paymentMethod сақталады, бұл болашақта Kaspi, bank card немесе басқа төлем провайдерімен қосылуға негіз болады. Төлем интеграциясы іске қосылған жағдайда webhook өңдеу, төлем қауіпсіздігі, транзакция тарихы және refund логикасы бөлек модуль ретінде жобалануы керек.

QR check-in функциясы да практикалық маңызы жоғары кеңейту бағыты. Қатысушы тіркелген соң QR код алса, іс-шара күні әкімші оны сканерлеп, checkedIn мәртебесін автоматты түрде белгілей алады. Бұл үлкен конференцияларда кезекті азайтады және нақты келушілер статистикасын жылдам жинауға мүмкіндік береді.

Аналитикалық модуль қосылған жағдайда әкімші қай іс-шараға қанша адам тіркелгенін, қандай залдар жиі брондалатынын, қандай формат сұранысқа ие екенін және өтінімдердің қандай бөлігі бекітілетінін көре алады. Мұндай деректер келесі іс-шараларды жоспарлауға, баға саясатын өзгертуге және зал ресурстарын тиімді бөлуге көмектеседі.

Жүйені дамытуда accessibility бағытына да көңіл бөлу қажет. Веб-интерфейс пернетақтамен басқаруға қолайлы болуы, түстер контрасты жеткілікті болуы, форма өрістерінде түсінікті label болуы және қате хабарламалары экран оқитын құралдарға да қолжетімді берілуі тиіс. WCAG қағидаларын ескеру жүйені тек техникалық тұрғыдан емес, әлеуметтік қолжетімділік тұрғысынан да сапалы етеді.

Интеграция мүмкіндіктері ORDA жүйесінің қолданылу аясын кеңейтеді. Болашақта Google Calendar немесе Outlook Calendar арқылы іс-шараны жеке күнтізбеге қосу, email арқылы растау хатын жіберу, Telegram немесе WhatsApp хабарламаларын қосу, CSV/Excel экспорт жасау және бухгалтерлік есеп

жүйелерімен байланысу сияқты функциялар енгізілуі мүмкін. Мұндай интеграциялар жүйені ұйымның басқа цифрлық құралдарымен байланыстырып, оның практикалық құндылығын арттырады.

Жобаның практикалық құндылығы оның аяқталған күйімен ғана емес, кеңейтуге дайын архитектурасымен де анықталады. Модульдік backend, бөлек frontend helper-лер, Mongoose модельдері және REST API келісімі жаңа функцияларды салыстырмалы түрде қауіпсіз қосуға мүмкіндік береді. Сондықтан ORDA жүйесін оқу жобасы ретінде ғана емес, нақты ұйымға бейімделетін бастапқы платформа ретінде қарастыруға болады.

ҚОРЫТЫНДЫ

Дипломдық жұмыста іс-шараларға тіркеу және конференц-залдарды басқаруға арналған ORDA Smart Event System веб-жүйесін әзірлеудің теориялық, жобалық және тәжірибелік аспектілері қарастырылды. Жұмыс барысында іс-шара ұйымдастыру процесін цифрландырудың маңызы, қатысушыларды тіркеу, залдарды брондау, өтінімдерді өңдеу және әкімшілік бақылау мәселелері талданды.

Бірінші бөлімде іс-шараларды басқару веб-жүйелерінің теориялық негіздері сипатталды. Қатысушы тәжірибесі, брондау процесі, ұқсас платформалар және қолданылған технологиялардың артықшылықтары қарастырылды. Бұл бөлім ORDA жүйесінің өзектілігін және жергілікті сценарийге бейімделу қажеттілігін негіздеді.

Екінші бөлімде жүйенің архитектурасы жобаланды. Пайдаланушы рөлдері, функционалдық талаптар, клиент-сервер байланысы, деректер қоры модельдері, API маршруттары және көптілді интерфейс шешімдері анықталды. MongoDB модельдері пайдаланушы, іс-шара, зал, өтінім және техникалық қолдау хабарламаларын бөлек сақтауға мүмкіндік берді.

Үшінші бөлімде жүйенің іске асырылуы мен тестілеуі сипатталды. Клиенттік бөлік HTML/CSS/JavaScript арқылы, серверлік бөлік Node.js және Express.js арқылы орындалды. Авторизацияда bcryptjs және JWT қолданылды, ал брондау логикасында орын саны мен уақыт қақтығысын тексеру енгізілді. Әкімші панелі статистика, өтінім мәртебесі, check-in, іс-шаралар және залдарды басқару мүмкіндіктерін қамтиды.

Осылайша, дипломдық жұмыстың мақсаты орындалды деп қорытынды жасауға болады. ORDA Smart Event System веб-жүйесі іс-шаралар мен конференц-залдарды басқаруды бір платформада біріктіреді, пайдаланушы мен әкімшіге түсінікті жұмыс сценарийін ұсынады және болашақта кеңейтуге дайын архитектураға ие.

Жұмыстың нәтижелері көрсеткендей, іс-шаралар мен конференц-залдарды басқару процесін цифрландыру ұйым үшін нақты практикалық нәтиже береді. Деректердің орталықтандырылуы, өтінім мәртебелерінің бақылануы, пайдаланушы рөлдерінің бөлінуі және әкімші панелі арқылы басқару бұрын бөлек жүргізілген әрекеттерді бір ақпараттық жүйеге біріктіреді. Бұл әсіресе оқу орындары мен шағын конференц-орталықтар үшін маңызды.

Теориялық бөлімде қарастырылған цифрландыру, пайдаланушы тәжірибесі, ұқсас платформалар және технологиялық стек мәселелері практикалық іске асырумен байланыстырылды. Eventbrite, Cvent және TimePad сияқты сервистерді талдау ORDA жүйесінің жергілікті сценарийге бейімделгенін және ішкі зал ресурстарын басқаруға бағытталғанын көрсетті. Бұл салыстыру жобаның нарықтағы орны мен даму шекарасын анықтауға мүмкіндік берді.

Жобалау бөлімінде клиент-сервер архитектурасы, REST API, MongoDB модельдері және рөлдік қолжетімділік негізделді. User, Event, Hall, Booking және SupportMessage коллекциялары жүйенің негізгі объектілерін сипаттайды,

ал controller және route қабаттары бизнес-логиканы нақты endpoint-терге бөледі. Бұл құрылым кодты түсінуге, өзгертуге және кеңейтуге ыңғайлы.

Іске асыру бөлімінде frontend пен backend байланысы нақты модульдер арқылы көрсетілді. Клиенттік бөлікте каталог, фильтрлер, брондау формасы, күнтізбе, карта, жеке кабинет, көптілділік және тақырып ауыстыру мүмкіндіктері қарастырылды. Серверлік бөлікте авторизация, брондау, уақыт қақтығысын анықтау, admin dashboard, check-in және support хабарламалары іске асырылды.

Қауіпсіздік тұрғысынан жүйеде парольді хэштеу, JWT авторизациясы, protected route, adminOnly middleware, CORS баптауы және environment variables қолданылды. Бұл шаралар production деңгейіндегі барлық талаптарды толық жабады деп айтуға болмайды, бірақ дипломдық жоба үшін қауіпсіз архитектураның негізгі қағидаларын көрсетеді және болашақта тереңдетуге дайын негіз қалыптастырады.

Тестілеу мен тәуекелдерді талдау жүйенің негізгі сценарийлері жұмыс істейтінін, бірақ әрі қарай дамытуда автоматты тесттер, accessibility аудит, төлем интеграциясы, хабарлама сервистері және аналитикалық есептер қажет болатынын көрсетті. Бұл қорытынды жобаның аяқталған нәтижесін ғана емес, оның келесі даму кезеңдерін де айқындайды.

Қорытындылай келе, ORDA Smart Event System дипломдық жұмыстың қойылған міндеттеріне сәйкес әзірленді және сипатталды. Жүйе іс-шараларды жариялау, қатысушыларды тіркеу, конференц-залдарды брондау, өтінімдерді өңдеу және әкімшілік бақылауды бір платформада біріктіреді. Оның басты артықшылығы - жергілікті ұйым қажеттіліктеріне бейімделуі, қолданылған технологиялардың ашықтығы, интерфейстің түсініктілігі және болашақта кеңейтуге дайын архитектурасы.

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР

1. Digital Kazakhstan | Қазақстан Республикасының Электрондық үкіметі. URL: <https://egov.kz/cms/kk/digital-kazakhstan>
2. ISO 9241-11:2018. Ergonomics of human-system interaction - Usability: Definitions and concepts. Geneva: ISO.
3. Nielsen J. Usability Engineering. San Francisco: Morgan Kaufmann, 1994.
4. Krug S. Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability. Berkeley: New Riders, 2014.
5. Pressman R. S., Maxim B. R. Software Engineering: A Practitioner's Approach. 9th ed. New York: McGraw-Hill, 2019.
6. Sommerville I. Software Engineering. 10th ed. Boston: Pearson, 2016.
7. MDN Web Docs. HTML, CSS and JavaScript documentation. URL: <https://developer.mozilla.org/>
8. Node.js Documentation. URL: <https://nodejs.org/en/docs>
9. Express.js Documentation. URL: <https://expressjs.com/>
10. MongoDB Manual. URL: <https://www.mongodb.com/docs/manual/>
11. Mongoose Documentation. URL: <https://mongoosejs.com/docs/>
12. JSON Web Token (JWT), RFC 7519. URL: <https://www.rfc-editor.org/rfc/rfc7519>
13. OWASP Foundation. Web Security Testing Guide. URL: <https://owasp.org/www-project-web-security-testing-guide/>
14. bcryptjs package documentation. URL: <https://www.npmjs.com/package/bcryptjs>
15. CORS package documentation. URL: <https://www.npmjs.com/package/cors>
16. Eventbrite. Event management platform. URL: <https://www.eventbrite.com/>
17. TimePad. Event registration and ticketing service. URL: <https://timepad.ru/>
18. Cvent. Event management software. URL: <https://www.cvent.com/>
19. W3C. Web Content Accessibility Guidelines (WCAG). URL: <https://www.w3.org/WAI/standards-guidelines/wcag/>
20. ORDA Smart Event System project files: backend/src, docs, package.json, DEFENSE_GUIDE.md, API.md.

ҚОСЫМША А. БАҒДАРЛАМА ҚҰРЫЛЫМЫ ЖӘНЕ НЕГІЗГІ КОД ҮЗІНДІЛЕРІ

Бұл қосымшада ORDA жобасының негізгі файлдық құрылымы және дипломдық жұмыста сипатталған модульдерге қатысты код үзінділері берілді. Толық бастапқы код жоба каталогында орналасқан.

8-кесте. Жоба файлдарының қысқаша сипаттамасы

Файл/каталог	Мазмұны
server.js	Жобаның негізгі іске қосу нүктесі, backend/server модулін шақырады
backend/src/app.js	Express қосымшасы, CORS, static files, health endpoint, API router
backend/src/routes	auth, user, event, hall, booking, admin, support маршруттары
backend/src/controllers	Бизнес-логика: тіркелу, логин, event, booking, hall, admin, support
backend/src/models	Mongoose модельдері: User, Event, Hall, Booking, SupportMessage
backend/src/utls/availability.js	Зал мен іс-шара уақыттарының қақтығысын анықтау логикасы
docs/index.html	Frontend view құрылымы және интерфейс блоктары
docs/app.js	Frontend негізгі логикасы, API шақырулары және UI жаңарту
docs/js/core/i18n.js	RU/KZ/EN сөздіктері және аударма функциялары

1-код үзіндісі. Health endpoint және API router қосылуы

```
app.get("/health", (req, res) => {
  res.json({
    status: "ok",
    brand: "ORDA Smart Event System",
    database: dbStatus(),
  });
});

app.use(createApiRouter());
app.use("/api", createApiRouter());
```

2-код үзіндісі. Пайдаланушыны тіркеу кезінде бірінші admin жасау логикасы

```
const adminsCount = await User.countDocuments({ role: "admin" });
const hash = await bcrypt.hash(password, 10);
const user = await User.create({
  name,
  email,
  password: hash,
  role: adminsCount === 0 ? "admin" : "user",
});
```

3-код үзіндісі. Іс-шараға тіркелу кезінде орын санын тексеру

```
const exists = await Booking.findOne({
  userEmail: req.user.email,
  eventId: event._id,
  status: { $in: activeBookingStatuses },
});
```



```
if (exists) {  
  throw new ApiError(400, "You are already registered for this event.");  
}  
  
const booked = await Booking.countDocuments({  
  eventId: event._id,  
  status: { $in: activeBookingStatuses },  
});  
  
if (booked >= event.capacity) {  
  throw new ApiError(400, "There are no free seats for this event.");  
}
```

4-код үзіндісі. Уақыт қақтығысын анықтау шарты

```
function overlaps(startA, endA, startB, endB) {  
  return startA < endB && endA > startB;  
}
```

ҚОСЫМША Б. ҚОЙЫЛАТЫН СКРИНШОТТАР ТІЗІМІ

Дипломдық жұмыста интерфейс суреттерін қолмен қою үшін арнайы орындар қалдырылды. Скриншоттарды жоба іске қосылғаннан кейін `http://localhost:3000` мекенжайынан түсіріп, төмендегі тізімге сәйкес тиісті орындарға орналастыру қажет.

9-кесте. Дипломға қойылатын скриншоттар тізімі

№	Скриншот атауы	Қай жерден түсіріледі
1	ORDA жүйесінің басты беті	Dashboard/Home view
2	Іс-шаралар каталогы және сүзгілер	Events view
3	Көптілділік және тақырып ауыстыру	Header language/theme controls
4	Жүйеге кіру және тіркелу модалы	Login/Register modal
5	Іс-шара карточкасы және тіркелу әрекеті	Event card/detail
6	Конференц-залдар каталогы	Halls view
7	Залды брондау өтінімі	Request/Hall booking form
8	Күнтізбе модулі	Calendar view
9	Floorplan немесе зал картасы	Floorplan view
10	Пайдаланушының жеке кабинеті	Profile view
11	Әкімші панелінің статистикасы	Admin dashboard
12	Әкімшінің іс-шара құру/өңдеу формасы	Admin event form
13	Әкімші өтінімдері және мәртебе өзгерту	Admin requests
14	Техникалық қолдау хабарламалары	Admin support messages